

Your Name Matthew Boiarski

COMP IV Sec 203: Project Portfolio

Spring 2023

Contents:

PS0 Hello World with SFML

PS1 Linear Feedback Shift Register and Image Encoding

PS1a LFSR

PS1b PhotMagic

PS2 Sokoban Game with SFML

PS2a Sokoban Part A

PS2b Sokoban Part B

PS3 Pythagoras Tree

PS4 Checkers Game with SFML

PS4a Checkers Part A

PS4b Checkers Part B

PS5 DNA Sequence Alignment

PS6 Random Writer

PS7 Kronos - Intro to Regular Expression

PSO Hello World with SFML

Program Outline:

In this assignment, I was introduced to the **Simple Fast Media Library**, or SFML for short and getting used to basic functionality with `sf::Sprite`.

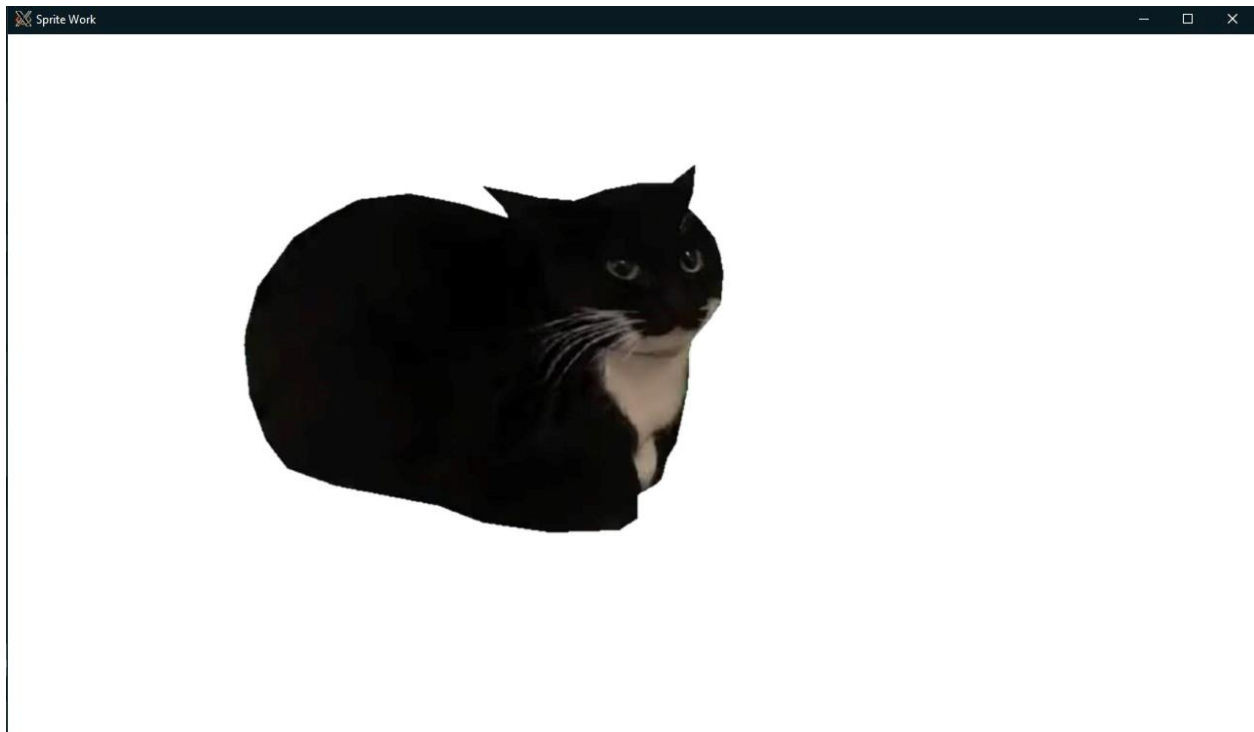
The overall function of this was to have some sort of playing field with a Sprite that holds a custom image of the programmer's choosing. After that, the programmer needed to implement a special function. In my case, I used two keys that either speeds up the Sprite's movement multiplier or slows it down.

The Sprite was the main component of the assignment.

I learned:

How to use SFML and the Sprite class, as well as being able to load an image with `sf::Image`.

Screenshot of Output:



main.cpp:

```

1  /*
2  * Name: Matthew Boiarski
3  * Course: COMP 2040 Computing IV
4  * Assignment #1: PS0
5  * Prof: Dr.Rykalova
6  * Date of Completion: 1/30/23
7  * Desc: This program is to demonstrate how to use the SFML
      library alongside linux (Hello World in this respect)
8  */
9  //debug
10 #include <iostream>
11 //SFML
12 #include <SFML/Graphics.hpp>
13 #include <SFML/Window.hpp>
14 #include <SFML/System.hpp>
15
16
17
18 //def const
19 #define WINDOW_WIDTH 1280      //PIX W
20 #define WINDOW_HEIGHT 720     //PIX H
21 #define SPRITE_WIDTH 50       //IMG CONTAINER W
22 #define SPRITE_HEIGHT 150     //IMG CONTAINER H
23 #define SIZE_MULTI 5          //SPRITE SIZE OFFSET
24 #define FPS 60                //FPS
25 #define V_SYNC false          //V_SYNC SETTING BOOL
26 #define IMAGE_FILE "sprite.png" //SPRITE IMAGE
27
28 //dec & init
29 int UNIT_MOVE = 10;           //UNITS OF MOVEMENT PER MOVE
30 float xDiff = 0.f, yDiff = 0.f;
31
32
33 //main
34 int main(int argc, char * argv[]) {
35     sf::VideoMode vid_mode;
36     vid_mode.width = WINDOW_WIDTH;
37     vid_mode.height = WINDOW_HEIGHT;
38     vid_mode.bitsPerPixel = 32;
39
40     //make window
41     sf::RenderWindow window(vid_mode, "Sprite Work", sf::Style::Resize | sf::Style::Close);
42
43     window.setFramerateLimit(FPS);
44     window.setVerticalSyncEnabled(V_SYNC);
45     //make texture to put on top of sprite
46
47

```

```

48 //make image (done for image flip later)
49 sf::Image image;
50 if (!image.loadFromFile(IMAGE_FILE)) {
51     std::cout << "ERROR: CANNOT LOAD IMAGE FILE!\n";
52     return -1;
53 }
54
55 //put image into texture
56 sf::Texture texture;
57 texture.loadFromImage(image);
58
59 //makes sprite
60 sf::Sprite sprite;
61 sprite.setTexture(texture);
62
63 while (window.isOpen()){
64     sf::Event event;
65     while (window.pollEvent(event)){
66         if (event.type == sf::Event::Closed)
67             window.close();
68
69         else if (event.type == sf::Event::Resized){
70
71             if (event.size.width < WINDOW_WIDTH || event
72                 .size.height < WINDOW_HEIGHT)
73             { window.setSize({ WINDOW_WIDTH,
74                 WINDOW_HEIGHT }); }
75
76             else if (event.size.width < WINDOW_WIDTH &&
77                 event.size.height < WINDOW_HEIGHT)
78             { window.setSize({ WINDOW_WIDTH,
79                 WINDOW_HEIGHT }); }
80
81         }
82     }
83     window.clear(sf::Color::White);
84
85     //keyboard
86     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left)
87         ||
88         sf::Keyboard::isKeyPressed(sf::Keyboard::A)) {
89         image.flipHorizontally();
90         xDiff = -UNIT_MOVE;
91     }
92     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right)
93         ||
94         sf::Keyboard::isKeyPressed(sf::Keyboard::D)) {
95         xDiff = UNIT_MOVE;
96     }
97     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up) ||
98         sf::Keyboard::isKeyPressed(sf::Keyboard::W)) {

```

```

92         yDiff = -UNIT_MOVE;
93     }
94     if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down)
95         ||
96         sf::Keyboard::isKeyPressed(sf::Keyboard::S)) {
97         yDiff = UNIT_MOVE;
98     }
99     if (sf::Keyboard::isKeyPressed(sf::Keyboard::R)) {
100         sprite.setPosition(0, 0); } //reset
101     if (sf::Keyboard::isKeyPressed(sf::Keyboard::F2)) {
102         UNIT_MOVE++; } //MAX IS FASTER
103     if (sf::Keyboard::isKeyPressed(sf::Keyboard::F3)) {
104         UNIT_MOVE--; } //MAX IS SLOWER
105
106     sprite.move(xDiff, yDiff);
107     xDiff = 0.f; yDiff = 0.f;
108     window.draw(sprite);
109     window.display();
110 }
111 return 0;
112 }

```

PS1A Linear Feedback Shift Register and Image Encoding Part A - LFSR

Program Outline:

I created a Fibonacci Feedback Shift Register and was able to step through it as needed for the step() and generate() function.

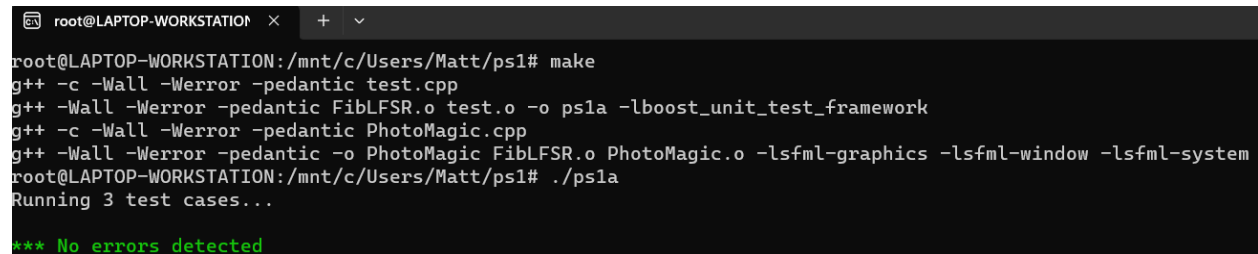
It uses tap points and does a chaining-XOR to the bits to determine the one to prepend.

Everything is working within an std::string. Added 3 BOOST tests to make sure my LFSR worked correctly.

I learned:

I got introduced to BOOST tests for the second time, first time was in COMP III, and got more adjusted to how the tests work, as well as becoming more professional with my code in general.

Screenshot of Output:

A terminal window titled 'root@LAPTOP-WORKSTATION' with a dark background. The terminal shows the following commands and output:

```
root@LAPTOP-WORKSTATION:/mnt/c/Users/Matt/ps1# make
g++ -c -Wall -Werror -pedantic test.cpp
g++ -Wall -Werror -pedantic FibLFSR.o test.o -o ps1a -lboost_unit_test_framework
g++ -c -Wall -Werror -pedantic PhotoMagic.cpp
g++ -Wall -Werror -pedantic -o PhotoMagic FibLFSR.o PhotoMagic.o -lsfml-graphics -lsfml-window -lsfml-system
root@LAPTOP-WORKSTATION:/mnt/c/Users/Matt/ps1# ./ps1a
Running 3 test cases...
*** No errors detected
```

Makefile:

```
1: CC = g++
2: CFLAGS = -c
3: WARNFLAGS = -Wall -Werror -pedantic
4: LIB = -lboost_unit_test_framework
5: all: ps1a
6: ps1a: FibLFSR.o test.o
7:      $(CC) $(WARNFLAGS) FibLFSR.o test.o -o ps1a $(LIB)
8: test.o: test.cpp
9:      $(CC) $(CFLAGS) $(WARNFLAGS) test.cpp
10: FibLFSR.o: FibLFSR.h
11:      $(CC) $(CFLAGS) $(WARNFLAGS) FibLFSR.cpp
12: clean:
13:      rm *.o ps1a
```

```

1 // Dr. Rykalova
2 // test.cpp for PS1a
3 // updated 1/31/2020
4
5 #include <iostream>
6 #include <string>
7
8 #include "FibLFSR.h"
9
10 #define BOOST_TEST_DYN_LINK
11 #define BOOST_TEST_MODULE Main
12 #include <boost/test/unit_test.hpp>
13 //checks the construct to see if the string gets flipped
    correctly
14 BOOST_AUTO_TEST_CASE(checkConstruct){
15     FibLFSR l("0000111100001111");
16     string flip = "1111000011110000";
17     BOOST_REQUIRE(flip.compare(l.getSeed()) == 0);
18 }
19 //runs step 10 times to make sure an empty reg always
    returns zero
20 BOOST_AUTO_TEST_CASE(zeroBitTen) {
21     FibLFSR l("0000000000000000");
22     BOOST_REQUIRE(l.step() == 0);
23     BOOST_REQUIRE(l.step() == 0);
24     BOOST_REQUIRE(l.step() == 0);
25     BOOST_REQUIRE(l.step() == 0);
26     BOOST_REQUIRE(l.step() == 0);
27     BOOST_REQUIRE(l.step() == 0);
28     BOOST_REQUIRE(l.step() == 0);
29     BOOST_REQUIRE(l.step() == 0);
30     BOOST_REQUIRE(l.step() == 0);
31     BOOST_REQUIRE(l.step() == 0);
32 }
33 BOOST_AUTO_TEST_CASE(sixteenBitsThreeTaps) {
34
35     FibLFSR l("1011011000110110");
36     BOOST_REQUIRE(l.step() == 0);
37     BOOST_REQUIRE(l.step() == 0);
38     BOOST_REQUIRE(l.step() == 0);
39     BOOST_REQUIRE(l.step() == 1);
40     BOOST_REQUIRE(l.step() == 1);
41     BOOST_REQUIRE(l.step() == 0);
42     BOOST_REQUIRE(l.step() == 0);
43     BOOST_REQUIRE(l.step() == 1);
44
45     FibLFSR l2("1011011000110110");
46     BOOST_REQUIRE(l2.generate(9) == 51);
47 }

```



```

1 #include <iostream>
2 #include <string>
3 #include <cmath>
4 using std::string;
5 using std::cin;
6 using std::cout;
7 using std::endl;
8 using std::ostream;
9 class FibLFSR {
10 public:
11     friend string strFlip(string);
12     FibLFSR(string str);
13     friend int XOR(int, int);
14     friend int char_to_val(char);
15     int step();
16     int generate(int);
17     string getSeed();
18     int getLength();
19     friend ostream& operator<<(ostream&, const FibLFSR&);
20
21 private:
22     string seed;
23     int length;
24 };

```

```

1 #include "FibLFSR.h"
2 //flips a string
3 string strFlip(string str) {
4     string res;
5     int strS = str.size() - 1;          //to set this strS as size
6     without the null term
7     if (strS == 0) { return ""; }      //catch case
8     else {
9         while (strS >= 0) {
10             res.push_back(str[strS]);
11             strS--;
12         }
13     }
14     return res;                        //returns non-terminated char
15                                     array stored in std::string
16 }
17 FibLFSR::FibLFSR(string str) {
18     if (str.size() != 16) { cout << "seed is size'" << str.
19         size() << "' when expected size is 16.\n"; }
20     else {
21         this->seed = strFlip(str);      //need to reverse string
22         this->length = 16;
23     }
24 }
25 //mimics logical XOR or Exclusive OR
26 int XOR(int a, int b) {
27     if ((a == 0 && b == 0) || (a == 1 && b == 1)) { return 0;
28     }
29     if ((a == 1 && b == 0) || (a == 0 && b == 1)) { return 1;
30     }
31     else {
32         return -1;
33     }
34 }
35 int char_to_val(char c) {
36     if (c == '0') { return 0; }
37     if (c == '1') { return 1; }
38     else{ return -1; } //forces 1 and 0
39 }
40 int FibLFSR::step() {
41     char end = this->seed.at(15);
42     char thirteen = this->seed.at(13);
43     char twelve = this->seed.at(12);
44     char ten = this->seed.at(10);
45     bool tap1 = XOR(char_to_val(end), char_to_val(thirteen));
46     bool tap2 = XOR(tap1, char_to_val(twelve));
47     bool res = XOR(tap2, char_to_val(ten));

```

```

45
46     if (res == 1) {
47         this->seed.pop_back();
48         this->seed = this->seed.insert(0, "1");
49     }
50     if (res == 0) {
51         this->seed.pop_back();
52         this->seed = this->seed.insert(0, "0");
53     }
54     return res;
55 }
56
57 int FibLFSR::generate(int k) {
58     int ans = 0;
59     for (k = k-1; k >= 0; k--) { //reverse loop because reg is
60         //flipped
61         if (this->step()) {
62             ans += pow(2, k);
63         }
64     }
65     return ans;
66 }
67
68 string FibLFSR::getSeed(){
69     return this->seed;
70 }
71
72 int FibLFSR::getLength(){
73     return this->length;
74 }
75 ostream& operator<<(ostream& out, const FibLFSR& fLFSR) {
76     out << strFlip(fLFSR.seed);
77     return out;
78 }
79 /*
80 int main(int argc, char* argv[]) {
81     int count = 0;
82     FibLFSR f("1011011000110110"); //def seed
83
84     //run step(); -> in loop now
85     cout << "Init: " << f << endl << endl;
86     //run generate();
87     cout << "K for generate: ";
88     cin >> count;
89     cout << f.generate(count);
90     return 0;
91 }*/

```

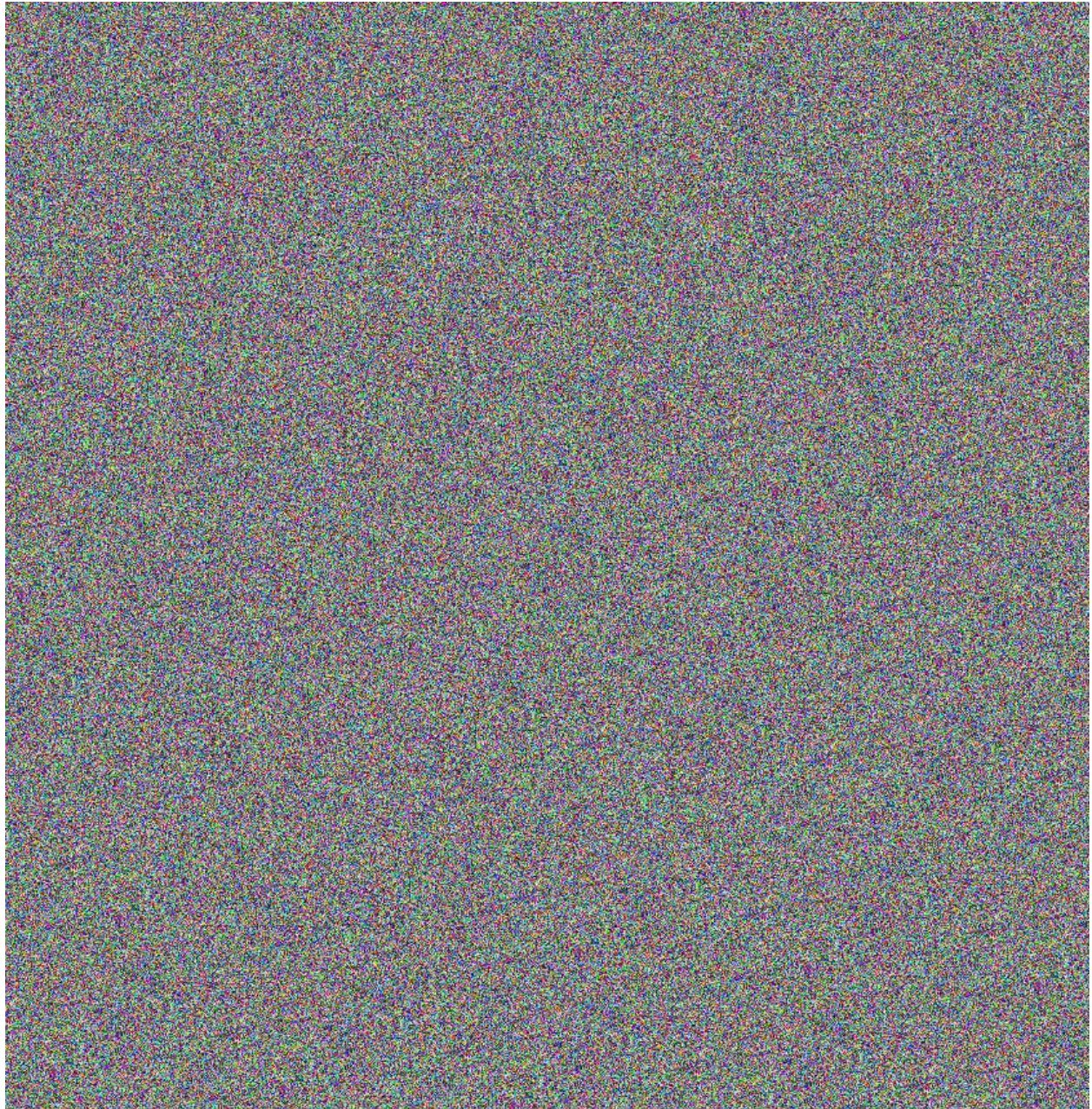

PS1B Linear Feedback Shift Register and Image Encoding Part B - PhotoMagic

Program Outline:

Now using the made LFSR, I was to take a .png file, and then scramble it pixel by pixel until the output .png file is created.

I learned: How data structures and files can be interrelated and can have useful functionality in tandem.

Screenshot of Output:



Makefile:


```
1: CC = g++
2: CFLAGS = -c
3: WARNFLAGS = -Wall -Werror -pedantic
4: BOOSTLIB = -lboost_unit_test_framework
5: SFMLLIB = -lsfml-graphics -lsfml-window -lsfml-system
6: all: ps1a PhotoMagic
7:
8: PhotoMagic: FibLFSR.o PhotoMagic.o
9:     $(CC) $(WARNFLAGS) -o PhotoMagic FibLFSR.o PhotoMagic.o $(SFMLLIB
)
10: PhotoMagic.o:
11:     $(CC) $(CFLAGS) $(WARNFLAGS) PhotoMagic.cpp
12: ps1a: FibLFSR.o test.o
13:     $(CC) $(WARNFLAGS) FibLFSR.o test.o -o ps1a $(BOOSTLIB)
14: test.o: test.cpp
15:     $(CC) $(CFLAGS) $(WARNFLAGS) test.cpp
16: FibLFSR.o: FibLFSR.h
17:     $(CC) $(CFLAGS) $(WARNFLAGS) FibLFSR.cpp
18: clean:
19:     rm *.o ps1a PhotoMagic
```

```

1 #include "FibLFSR.h"
2 #include <SFML/Graphics.hpp>
3 #define IMG_OFFSET 25
4 void transform(sf::Image &img, FibLFSR fibL)
5 {
6     sf::Color color;
7     sf::Vector2u size = img.getSize();
8
9     for (uint x = 0; x < size.x; x++)
10    {
11        for (uint y = 0; y < size.y; y++)
12        {
13            color = img.getPixel(x, y); // gets r,g,b and A
14                                     value
15            color.r ^= fibL.generate(8);
16            color.g ^= fibL.generate(8);
17            color.b ^= fibL.generate(8);
18            img.setPixel(x, y, color);
19        }
20    }
21 }
22 int main(int argc, char *argv[]){
23     sf::Image image, output;
24     FibLFSR f(argv[3]);
25
26     if (!image.loadFromFile(argv[1]))
27         return -1;
28     output = image;
29     transform(output, f);
30
31     sf::Vector2u bounds = image.getSize();
32     sf::RenderWindow window(sf::VideoMode(bounds.x +
33                                     IMG_OFFSET, bounds.y + IMG_OFFSET), "PhotoMagic:
34                                     Original Image");
35     sf::RenderWindow window_out(sf::VideoMode(bounds.x +
36                                     IMG_OFFSET, bounds.y + IMG_OFFSET), "PhotoMagic:
37                                     Transformed Image");
38     sf::Texture hold_input, hold_output;
39     hold_input.loadFromImage(image);
40     hold_output.loadFromImage(output);
41
42     sf::Sprite sprite_in, sprite_out;
43     sprite_in.setTexture(hold_input);
44     sprite_out.setTexture(hold_output);
45
46     sprite_out.move(bounds.x, 0);
47
48     while (window.isOpen() && window_out.isOpen())

```

```

45     {
46         sf::Event event;
47
48         while (window.pollEvent(event))
49         {
50             if (event.type == sf::Event::Closed)
51                 window.close();
52         }
53
54         while (window_out.pollEvent(event))
55         {
56             if (event.type == sf::Event::Closed)
57                 window_out.close();
58         }
59
60         window.clear(sf::Color::White);
61         window.draw(sprite_in);
62         window.display();
63
64         window_out.clear(sf::Color::White);
65         window_out.draw(sprite_out);
66         window_out.display();
67     }
68
69     if (!output.saveToFile(argv[2])){
70         return -1;
71     }
72     return 0;
73 }

```

```

1 #include <iostream>
2 #include <string>
3 #include <cmath>
4 using std::string;
5 using std::cin;
6 using std::cout;
7 using std::endl;
8 using std::ostream;
9 class FibLFSR {
10 public:
11     friend string strFlip(string);
12     FibLFSR(string str);
13     friend int XOR(int, int);
14     friend int char_to_val(char);
15     int step();
16     int generate(int);
17     string getSeed();
18     int getLength();
19     friend ostream& operator<<(ostream&, const FibLFSR&);
20
21 private:
22     string seed;
23     int length;
24 };

```



```

1 #include "FibLFSR.h"
2 //flips a string
3 string strFlip(string str) {
4     string res;
5     int strS = str.size() - 1;          //to set this strS as size
6     without the null term
7     if (strS == 0) { return ""; }      //catch case
8     else {
9         while (strS >= 0) {
10             res.push_back(str[strS]);
11             strS--;
12         }
13     }
14     return res;                        //returns non-terminated char
15                                     array stored in std::string
16 }
17 FibLFSR::FibLFSR(string str) {
18     if (str.size() != 16) { cout << "seed is size'" << str.
19         size() << "' when expected size is 16.\n"; }
20     else {
21         this->seed = strFlip(str);      //need to reverse string
22         this->length = 16;
23     }
24 }
25 //mimics logical XOR or Exclusive OR
26 int XOR(int a, int b) {
27     if ((a == 0 && b == 0) || (a == 1 && b == 1)) { return 0;
28     }
29     if ((a == 1 && b == 0) || (a == 0 && b == 1)) { return 1;
30     }
31     else {
32         return -1;
33     }
34 }
35 int char_to_val(char c) {
36     if (c == '0') { return 0; }
37     if (c == '1') { return 1; }
38     else{ return -1; } //forces 1 and 0
39 }
40 int FibLFSR::step() {
41     char end = this->seed.at(15);
42     char thirteen = this->seed.at(13);
43     char twelve = this->seed.at(12);
44     char ten = this->seed.at(10);
45     bool tap1 = XOR(char_to_val(end), char_to_val(thirteen));
46     bool tap2 = XOR(tap1, char_to_val(twelve));
47     bool res = XOR(tap2, char_to_val(ten));

```

```

45
46     if (res == 1) {
47         this->seed.pop_back();
48         this->seed = this->seed.insert(0, "1");
49     }
50     if (res == 0) {
51         this->seed.pop_back();
52         this->seed = this->seed.insert(0, "0");
53     }
54     return res;
55 }
56
57 int FibLFSR::generate(int k) {
58     int ans = 0;
59     for (k = k-1; k >= 0; k--) { //reverse loop because reg is
60         //flipped
61         if (this->step()) {
62             ans += pow(2, k);
63         }
64     }
65     return ans;
66 }
67
68 string FibLFSR::getSeed(){
69     return this->seed;
70 }
71
72 int FibLFSR::getLength(){
73     return this->length;
74 }
75 ostream& operator<<(ostream& out, const FibLFSR& fLFSR) {
76     out << strFlip(fLFSR.seed);
77     return out;
78 }
79 /*
80 int main(int argc, char* argv[]) {
81     int count = 0;
82     FibLFSR f("1011011000110110"); //def seed
83
84     //run step(); -> in loop now
85     cout << "Init:  " << f << endl << endl;
86     //run generate();
87     cout << "K for generate: ";
88     cin >> count;
89     cout << f.generate(count);
90     return 0;
91 }*/

```

PS2a Sokoban

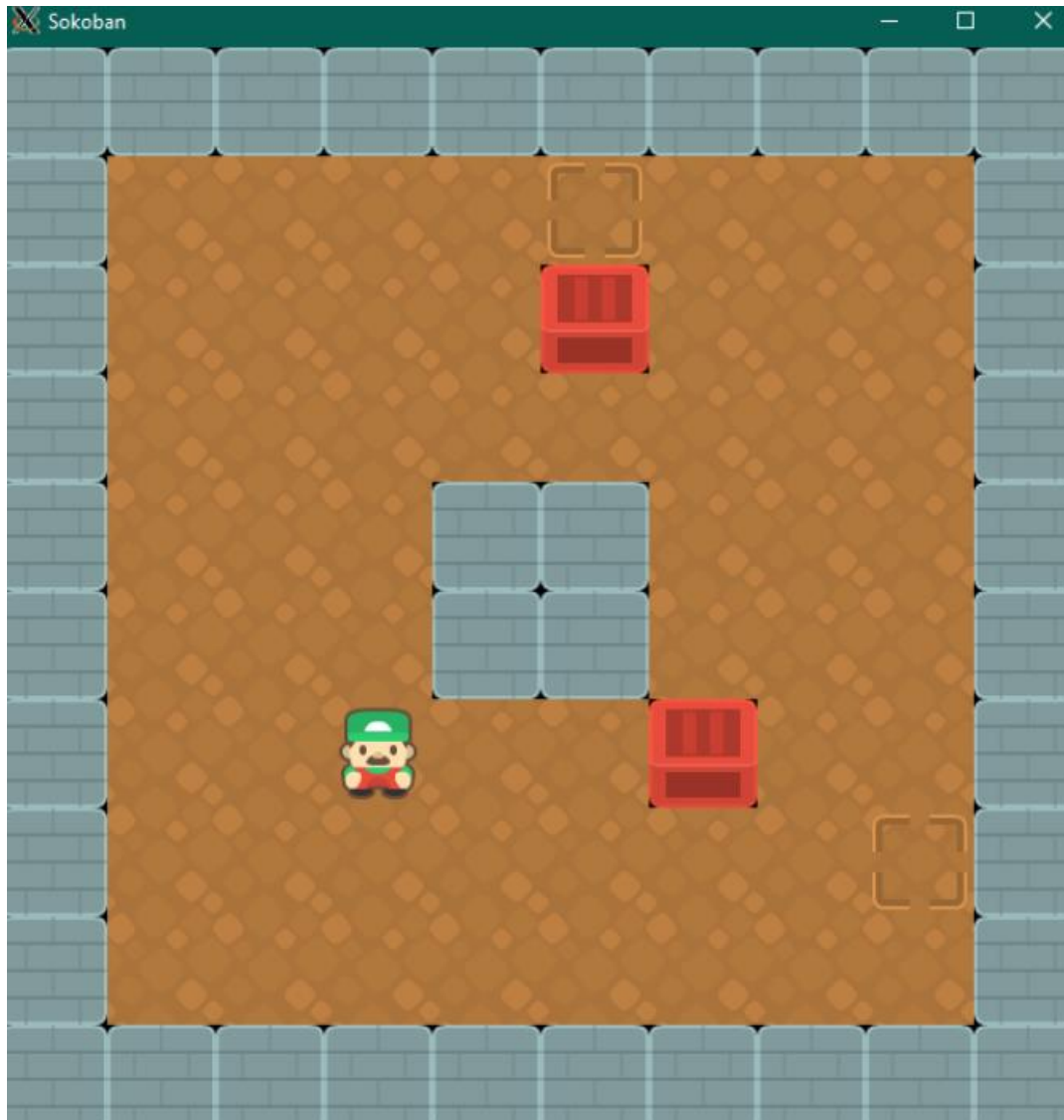
Program Outline:

In this program, I was supposed to create a classic Sokoban-style game in which I needed to again create a movable `sf::Sprite`, but with a background, and a grid-like gameboard that consisted of either empty spaces, walls, moveable blocks, and chests. Since this is part A, I only needed to create a program that was supposed to read in a `.lvl` or `.txt` file to set the gameboard following a character legend that was provided in the assignment instructions. I was to again use the SFML.

I learned: How to be able to make a simple gameboard within C++ and SFML.

At first, I was not successful in the project shown, however, I was able to get it to work correctly in the second PSX assignment.

Screenshot of Output:



```
1: CC = g++
2: CFLAGS = -std=c++11 -g -Og -Wall -Werror -pedantic
3: SFMLFLAGS = -lsfml-graphics -lsfml-window -lsfml-system
4:
5: all: Sokoban Sokoban.o main.o
6:
7: Game: Sokoban.o main.o
8:      $(CC) $(CFLAGS) -o Sokoban Sokoban.o main.o $(SFMLFLAGS)
9:
10: main.o: main.cpp Sokoban.h
11:      $(CC) $(CFLAGS) -c main.cpp -o main.o
12:
13: Sokoban.o: Sokoban.cpp
14:      $(CC) $(CFLAGS) -c Sokoban.cpp -o Sokoban.o
15:
16: lint:
17:      cpplint --filter=--runtime/references,-build/c++11 --root=. main.c
pp Sokoban.cpp Sokoban.h
18: clean:
19:      rm Sokoban
20:      rm *.o
```

```

1 //Copyright Matthew Boiarski 2023
2 #include "Sokoban.h"
3 int main(int argc, char* argv[]) {
4     if (argc != 2) {
5         cout << "ERROR W/ argc!\n ";
6         return -1;
7     }
8     std::ifstream test;
9     test.open(argv[1]);
10    if (!test) {
11        cout << "ERROR W/ IFSTREAM!\n";
12        return -1;
13    }
14    Sokoban soko;
15    test >> soko;
16    sf::RenderWindow window(
17        sf::VideoMode(soko.getWidth() * 16, soko.getHeight()
18            * 16),
19        "Sokoban");
20    sf::Time SFML_TIME;
21    sf::Clock clock;
22    while (window.isOpen()) {
23        sf::Event event;
24        while (window.pollEvent(event)) {
25            if (event.type == sf::Event::Closed) window.
26                close();
27        }
28        window.clear(sf::Color::White);
29        window.draw(soko);
30        window.display();
31        SFML_TIME = clock.getElapsedTime();
32        //cout << "Time (in millis): " << SFML_TIME.
33            asMilliseconds() << endl;
34    }
35    return 0;
36 }

```

```

1 //Copyright Matthew Boiarski 2023
2 #include <iostream>
3 #include <algorithm>
4 #include <fstream>
5 #include <vector>
6 #include <string>
7 #include <SFML/Graphics.hpp>
8
9 using std::ostream;
10 using std::istream;
11 using std::cout;
12 using std::endl;
13 using std::vector;
14 using std::string;
15 using std::for_each;
16 #define CORD_IT 64
17
18 enum Direction {
19     LEFT, RIGHT, UP, DOWN
20 };
21
22 class Cell : public sf::Sprite {
23 public:
24     friend class Sokoban;
25     int xCORD, yCORD;
26 };
27
28 class Sokoban : public sf::Drawable {
29 public:
30     Sokoban();
31     ~Sokoban();
32     //void movePlayer(Direction d);
33     int getWidth() { return w; }
34     int getHeight() { return h; }
35 private:
36     virtual void draw(sf::RenderTarget&, sf::RenderStates)
37         const;
38     int w, h;
39     sf::Texture boxes, emptys, walls, storages, /*Player imgs
40         */ pUP, pDOWN, pLEFT, pRIGHT;
41     sf::Sprite hold_screen;
42
43     //holds map of game in a single vec of type Cell (x, y,
44         sf::Texture)
45     vector<Cell> gameMap;
46     string gameMap_string; //string rep
47
48     //streams
49     friend ostream& operator<<(ostream&, const Sokoban&);

```

```
47 |     friend istream& operator>>(istream&, Sokoban&);  
48 | };
```

```

1 //Copyright Matthew Boiarski 2023
2 #include "Sokoban.h"
3
4 Sokoban::Sokoban() {
5     sf::Image image;
6     //player-specific images
7     image.loadFromFile("player_05.png");
8     pUP.loadFromImage(image);
9     image.loadFromFile("player_17.png");
10    pDOWN.loadFromImage(image);
11    image.loadFromFile("player_08.png");
12    pLEFT.loadFromImage(image);
13    image.loadFromFile("block_06.png");
14    pRIGHT.loadFromImage(image);
15    //non-player images
16    image.loadFromFile("player_20.png");
17    walls.loadFromImage(image);
18    image.loadFromFile("crate_03.png");
19    boxes.loadFromImage(image);
20    image.loadFromFile("ground_01.png");
21    emptys.loadFromImage(image);
22    image.loadFromFile("ground_04.png");
23    storages.loadFromImage(image);
24 }
25
26 void Sokoban::draw(sf::RenderTarget& tar, sf::RenderStates
27     render) const {
28     tar.draw(hold_screen);
29     for_each(gameMap.begin(),
30         gameMap.end(),
31         [&tar](Cell c)
32         {tar.draw(c); });
33 }
34 istream& operator>>(istream& in, Sokoban& soko) {
35     //clear-loop
36     while (!soko.gameMap.empty()) {
37         soko.gameMap.pop_back();
38     }
39     int h, w;
40     in >> h >> w;
41     soko.h = h; soko.w = w;
42     in >> soko.gameMap_string;
43     char c;
44     int buffer_pos = 0;
45     while (in.get(c)) {
46         if (c == '\n') continue;
47         Cell sprite;
48         if(c == '#') { sprite.setTexture(soko.walls); }

```



```

49         if(c == 'a'){ sprite.setTexture(soko.storages); }
50         if(c == 'A'){ sprite.setTexture(soko.bboxes); }
51         if(c == '@'){ sprite.setTexture(soko.pRIGHT); }
52         buffer_pos++;
53         sprite.setPosition(CORD_IT * (buffer_pos % w),
54                             CORD_IT * (buffer_pos / w));
55         sprite.xCORD = buffer_pos % w;
56         sprite.yCORD = buffer_pos / w;
57         soko.gameMap.push_back(sprite);
58     }
59     soko.empty.setRepeated(true);
60     soko.hold_screen.setTexture(soko.empty);
61     soko.hold_screen.setTextureRect(sf::IntRect(0, 0,
62         CORD_IT * w, CORD_IT * h));
63     return in;
64 }
65 ostream& operator<<(ostream& out, const Sokoban& soko) {
66     out << "W:" << soko.w << "\t" << "H: " << soko.h;
67     out << "\tGAMEMAP: " << soko.gameMap_string;
68     return out;
69 }

```

PS2b Sokoban

Program Outline:

Following the last program to build from it, I needed to add game rules, including making the block movable, making the storage work, as well as a game end message.

I learned: How to be able to make a simple gameboard within C++ and SFML.

Lambda has also been added in.

At first, I was not successful in the project shown, however, I was able to get it to work correctly in the second PSX assignment.

Screenshot of Output:



```
1: CC = g++
2: CFLAGS = --std=c++17 -Wall -Werror -pedantic
3: LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -lboost_un
it_test_framework
4: OBJ = main.o sokoban.o
5:
6: .PHONY: all clean lint
7:
8: all: Sokoban lint
9:
10: Game: $(OBJ)
11:      $(CC) $(CFLAGS) -o $@ $^ $(LIB)
12:
13: %.o: %.cpp $(LIB)
14:      $(CC) $(CFLAGS) -c $<
15:
16: clean:
17:      rm *.o
18:      rm Sokoban
19:
20: lint:
21:      cpplint *.cpp *.hpp
```

```

1 // Copyright Matthew Boiarski 2023
2 #include "sokoban.hpp"
3
4 int main(int argc, char * argv[]) {
5     ifstream myIn(argv[1]);
6     Sokoban game;
7     myIn >> game;
8     RenderWindow gameWindow
9     (VideoMode(game.getWidth()*CONVERT, game.getHeight()*CONVERT
10        ), "Sokoban");
11     while (gameWindow.isOpen()) {
12         Event event;
13         while (gameWindow.pollEvent(event)) {
14             if (event.type == Event::Closed) {
15                 gameWindow.close();
16             } else if (event.type == Event::KeyPressed) {
17                 if (event.key.code == Keyboard::W) {
18                     game.movePlayer(UP);
19                 } else if (event.key.code == Keyboard::S) {
20                     game.movePlayer(DOWN);
21                 } else if (event.key.code == Keyboard::A) {
22                     game.movePlayer(LEFT);
23                 } else if (event.key.code == Keyboard::D) {
24                     game.movePlayer(RIGHT);
25                 } else if (event.key.code == Keyboard::R) {
26                     myIn.clear();
27                     myIn.seekg(0, ios::beg);
28                     myIn >> game;
29                 }
30             }
31             gameWindow.clear(Color::White);
32             gameWindow.draw(game);
33             gameWindow.display();
34         }
35     }

```

```

1 // Copyright Matthew Boiarski 2023
2 #ifndef SOKOBAN_HPP
3 #define SOKOBAN_HPP
4 #define CONVERT 64
5 #include <iostream>
6 #include <algorithm>
7 #include <fstream>
8 #include <vector>
9 #include <SFML/Graphics.hpp>
10 using std::istream;
11 using std::ifstream;
12 using std::ostream;
13 using std::cout;
14 using std::endl;
15 using std::vector;
16 using std::ios;
17 using sf::IntRect;
18 using sf::Event;
19 using sf::Keyboard;
20 using sf::Drawable;
21 using sf::Image;
22 using sf::Texture;
23 using sf::RenderTexture;
24 using sf::Color;
25 using sf::Sprite;
26 using sf::VideoMode;
27 using sf::RenderWindow;
28 using sf::RenderTarget;
29 using sf::RenderStates;
30
31 enum MoveD {
32     RIGHT, LEFT, UP, DOWN
33 };
34
35 class Sokoban;
36
37 ostream& operator<<(ostream&, const Sokoban&);
38
39 class GameBlock : public Sprite {
40     private:
41     char hold;
42     friend class Sokoban;
43     friend istream& operator>>(istream&, Sokoban&);
44 };
45
46 class Sokoban : public Drawable {
47     public:
48     Sokoban();
49     void movePlayer(MoveD);

```

```

50| int getWidth();
51| int getHeight();
52| private:
53| virtual void draw(RenderTarget&, RenderStates) const;
54| int W, H, playIt;
55| int counter;
56| bool ifWin = false;
57| RenderTexture rTexture;
58| vector <GameBlock> gamboard;
59| Texture
60| box, empty, solid, chest,
61| faceL, faceR, faceU, faceD;
62| Sprite player;
63| friend ostream& operator<<(ostream&, const Sokoban&);
64| friend istream& operator>>(istream&, Sokoban&);
65| };
66| istream& operator>>(istream& os, Sokoban&);
67| #endif // !SOKOBAN_HPP

```

```

1 // Copyright Matthew Boiarski 2023
2 #include "sokoban.hpp"
3
4 Sokoban::Sokoban() {
5     Image img;
6     rTexture.create(CONVERT, CONVERT);
7     counter = 0;
8     img.loadFromFile("./sokoban/player_05.png");
9     faceD.loadFromImage(img);
10    img.loadFromFile("./sokoban/player_08.png");
11    faceU.loadFromImage(img);
12    img.loadFromFile("./sokoban/player_17.png");
13    faceR.loadFromImage(img);
14    img.loadFromFile("./sokoban/player_20.png");
15    faceL.loadFromImage(img);
16    img.loadFromFile("./sokoban/block_06.png");
17    solid.loadFromImage(img);
18    img.loadFromFile("./sokoban/crate_03.png");
19    box.loadFromImage(img);
20    img.loadFromFile("./sokoban/ground_01.png");
21    empty.loadFromImage(img);
22    img.loadFromFile("./sokoban/ground_04.png");
23    chest.loadFromImage(img);
24 }
25
26 void Sokoban::draw(RenderTarget& target, RenderStates states
27     ) const {
28     target.draw(player);
29     for_each(gamboard.begin(), gamboard.end(),
30         [&target](GameBlock t) {target.draw(t);});
31 }
32
33 istream& operator>>(istream& in, Sokoban& obj) {
34     in >> obj.H >> obj.W;
35     char c;
36     int it = -1;
37     obj.gamboard.clear();
38     obj.ifWin = false;
39     while (in.get(c)) {
40         if (c == '\n') continue;
41         it++;
42         GameBlock sprite;
43         sprite.hold = c;
44         switch (c) {
45             case '#':
46                 sprite.setTexture(obj.solid);
47                 break;
48             case 'a':
49                 obj.counter++;

```

```

49 sprite.setTexture(obj.chest);
50 break;
51 case 'A':
52 sprite.setTexture(obj.box);
53 break;
54 case '@':
55 sprite.setTexture(obj.faceD);
56 obj.playIt = it;
57 break;
58 }
59 sprite.setPosition(CONVERT * (it % obj.W), CONVERT * (it /
    obj.W));
60 obj.gamboard.push_back(sprite);
61 }
62 obj.empty.setRepeated(true);
63 obj.player.setTexture(obj.empty);
64 obj.player.setTextureRect(IntRect(0, 0, CONVERT * obj.W,
    CONVERT * obj.H));
65 return in;
66 }
67
68 void Sokoban::movePlayer(MoveD d) {
69 if (ifWin) return; // bring to end to give instant win
70 int nowP = (d == DOWN) ? W : ((d == UP) ? -W : (d == RIGHT)
    ? 1 : -1);
71 if (gamboard[playIt + nowP].hold == '.' &&
72 (((d == UP || d == DOWN) || (playIt + nowP) / W == (playIt /
    W)))) {
73 if (gamboard[playIt].hold == '@') {
74 gamboard[playIt].hold = '.';
75 gamboard[playIt].setTexture(empty);
76 } else if (gamboard[playIt].hold == 'D') {
77 gamboard[playIt].hold = 'a'; gamboard[playIt].setTexture
    (chest);
78 }
79 playIt += nowP;
80 if (d == LEFT) gamboard[playIt].setTexture(faceL);
81 if (d == RIGHT) gamboard[playIt].setTexture(faceR);
82 if (d == DOWN) gamboard[playIt].setTexture(faceD);
83 if (d == UP) gamboard[playIt].setTexture(faceU);
84 gamboard[playIt].hold = '@';
85 } else if (gamboard[playIt + nowP].hold == 'A' ||
86 gamboard[playIt + nowP].hold == 'S') {
87 if ((d != RIGHT && d != LEFT) ||
88 ((playIt + ((d == RIGHT) ? 2 : -2)) / W == playIt / W)) {
89 if (gamboard[playIt + nowP + nowP].hold == '.' ||
90 gamboard[playIt + nowP + nowP].hold == 'a') {
91 if (gamboard[playIt].hold == '@') {
92 gamboard[playIt].setTexture(empty); gamboard[playIt].hold =
    '.';

```



```

93 } else if (gamboard[playIt].hold == 'D') {
94     gamboard[playIt].setTexture(chest); gamboard[playIt].hold =
        'a';
95 }
96
97 playIt+= nowP;
98
99 if (gamboard[playIt].hold == 'A') {
100     gamboard[playIt].hold = '@';
101     if (d == LEFT) gamboard[playIt].setTexture(faceL);
102     else if (d == RIGHT) gamboard[playIt].setTexture(faceR);
103     else if (d == DOWN) gamboard[playIt].setTexture(faceD);
104     else if (d == UP) gamboard[playIt].setTexture(faceU);
105     if (gamboard[playIt].hold == 'S') {
106         gamboard[playIt].hold = 'D';
107         rTexture.clear(Color::White);
108         rTexture.draw(Sprite(chest));
109     }
110     if (d == LEFT) rTexture.draw(Sprite(faceL));
111     else if (d == RIGHT) rTexture.draw(Sprite(faceR));
112     else if (d == DOWN) rTexture.draw(Sprite(faceD));
113     else if (d == UP) rTexture.draw(Sprite(faceU));
114     rTexture.display();
115     gamboard[playIt].setTexture(rTexture.getTexture());
116     counter++;}
117 if (gamboard[playIt + nowP].hold == 'a') {
118     counter--;
119     gamboard[playIt + nowP].hold = 'S';
120 } else {
121     gamboard[playIt + nowP].hold = 'A';
122 }
123 gamboard[playIt + nowP].setTexture(box);
124 }
125 }
126 } else if (gamboard[playIt + nowP].hold == 'a') {
127     if (gamboard[playIt].hold == '@') {
128         gamboard[playIt].hold = '.'; gamboard[playIt].setTexture(
            empty);
129     } else if (gamboard[playIt].hold == 'D') {
130         gamboard[playIt].hold = 'a'; gamboard[playIt].setTexture(
            chest);
131     }
132     playIt+= nowP;
133     rTexture.clear(Color::White);
134     rTexture.draw(Sprite(chest));
135     if (d == LEFT) rTexture.draw(Sprite(faceL));
136     else if (d == RIGHT) rTexture.draw(Sprite(faceR));
137     else if (d == DOWN) rTexture.draw(Sprite(faceD));
138     else if (d == UP) rTexture.draw(Sprite(faceU));
139     rTexture.display();

```

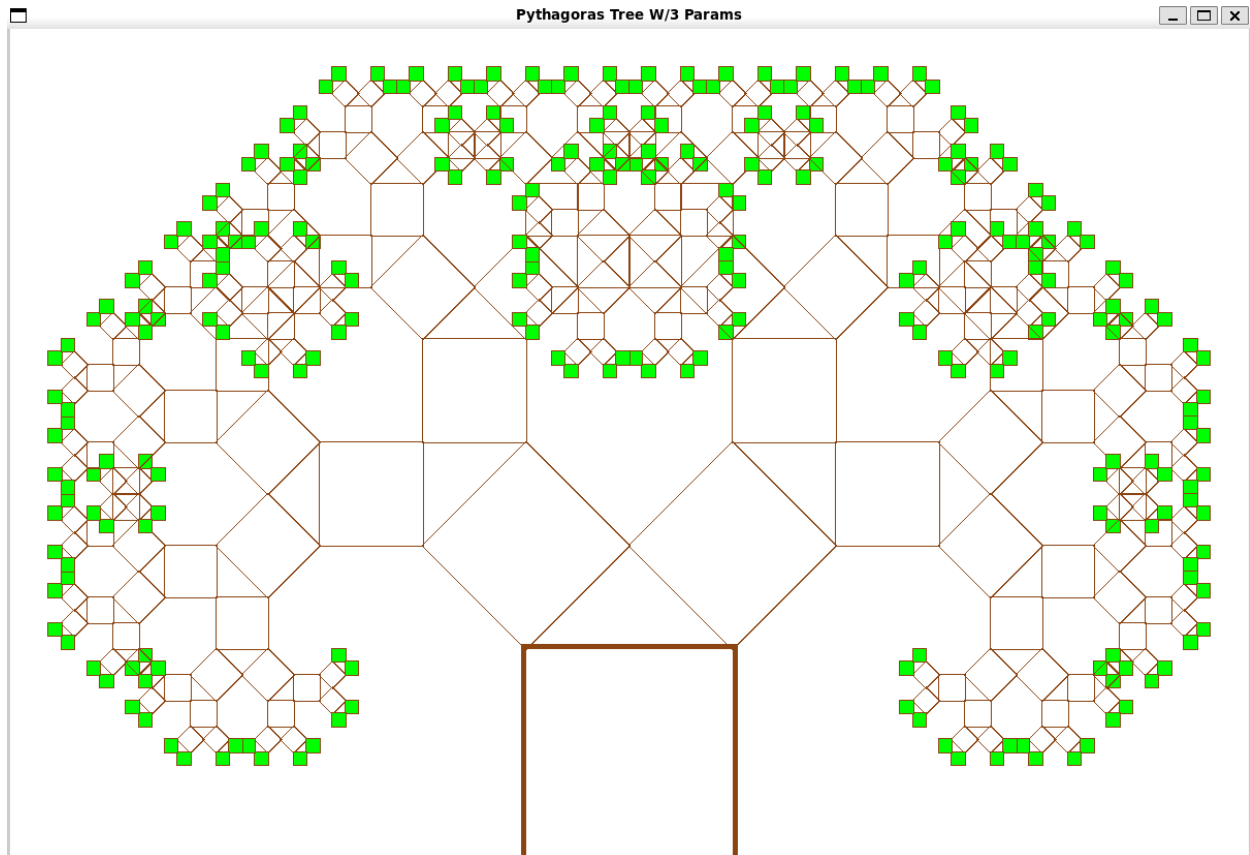
```
140 gamboard[playIt].setTexture(rTexture.getTexture());
141 gamboard[playIt].hold = 'D';
142 }
143 if (counter == 0) {
144     cout << "You win!" << endl;
145     ifWin = true;
146 }
147 }
148
149 int Sokoban::getWidth() { return W; }
150 int Sokoban::getHeight() { return H; }
```

PS3 Pythagoras Tree

Program Outline: In this program, I was to be able to display a Pythagoras Tree using recursive graphics with SFML. I used a recursive statement in the draw() function, and I managed to also achieve the extra credit which includes a 3rd parameter in the constructor for PTree, or Theta.

I learned: How to be able to make a recursive graphic program using a popular fractal pattern including a mathematically accurate Pythagoras Tree.

Screenshot of Output:



```
1: CC = g++
2: CFLAGS = -std=c++11 -g -Wall -Werror -pedantic
3: SFMLFLAGS = -lsfml-graphics -lsfml-window -lsfml-system
4:
5: all: lint PTree PTree.o main.o
6:
7: PTree: main.o PTree.o
8:     $(CC) $(CFLAGS) -o PTree main.o PTree.o $(SFMLFLAGS)
9: main.o: main.cpp PTree.h
10:     $(CC) $(CFLAGS) -c main.cpp -o main.o $(SFMLFLAGS)
11: PTree.o: PTree.cpp PTree.h
12:     $(CC) $(CFLAGS) -c PTree.cpp -o PTree.o $(SFMLFLAGS)
13: lint:
14:     cpplint --filter=--runtime/references,-build/c++11 --root=. main.c
pp PTree.cpp PTree.h
15: clean:
16:     rm *.o
17:     rm PTree
```

```

1 // Copyright Matthew Boiarski 2023
2 #include "PTree.h"
3 #include <SFML/Graphics/RectangleShape.hpp>
4 int main(int argc, char * argv[]) {
5     int N, len;
6     double L, T;
7     if (argc == 1) cout << "NO PARAMS!\t ENTER YOUR INPUT AS: '
        double, int'\n";
8     if (argc == 2) cout << "ONE PARAM!\t ENTER YOUR INPUT AS: '
        double, int'\n";
9     if (argc == 3) {
10 L = atof(argv[1]);
11 len = static_cast<int>(L);
12 N = atoi(argv[2]);
13 PTree tree(N, L);
14 // cout << tree;
15 RenderWindow window(VideoMode(6 * len, 4 * len), "PTree W/2
        Params");
16 while (window.isOpen()) {
17     Event event;
18     while (window.pollEvent(event)) {
19         if (event.type == Event::Closed)
20             window.close();
21         break;
22     }
23     window.clear(Color::White);
24     window.draw(tree);
25     window.display();
26 }
27 }
28 if (argc == 4) {
29 L = atof(argv[1]);
30 len = static_cast<int>(L);
31 N = atoi(argv[2]);
32 T = atof(argv[3]);
33 PTree tree(N, L, T);
34 // cout << tree;
35 RenderWindow window(VideoMode(6 * len, 4 * len), "Pythagoras
        Tree W/3 Params");
36 while (window.isOpen()) {
37     Event event;
38     while (window.pollEvent(event)) {
39         if (event.type == Event::Closed)
40             window.close();
41         break;
42     }
43     window.clear(Color::White);
44     window.draw(tree);
45     window.display();

```

```
46 | }  
47 | }  
48 | return 0;  
49 | }
```

```

1 #ifndef PTREE_H_
2 #define PTREE_H_
3 // Copyright Matthew Boiarski 2023
4 #define _USE_MATH_DEFINES
5 #include <iostream>
6 #include <utility>
7 #include <vector>
8 // gave a compile issue when using cmath alone
9 #include <cmath>
10 #include <SFML/Graphics.hpp>
11 #include <SFML/Graphics/RectangleShape.hpp>
12 #define THETA 30
13 // flag
14 #define SIDE_LOW 1.f
15
16 using std::vector;
17 using sf::RectangleShape;
18 using std::cin;
19 using std::cout;
20 using std::endl;
21 using std::istream;
22 using std::ostream;
23 using std::begin;
24 using std::end;
25 using sf::Drawable;
26 using sf::Texture;
27 using sf::Event;
28 using sf::RenderStates;
29 using sf::RenderTarget;
30 using sf::RenderWindow;
31 using sf::Color;
32 using sf::View;
33 using sf::VideoMode;
34 using sf::Vector2;
35 using sf::Vector2f;
36 using sf::Vector2u;
37 // class definition
38 class PTree : public Drawable{
39 private:
40 double
41 // base Length or initial value of L
42 base_L,
43 // Angle between base square and next tree squares
44 // (sin(theta) * L or cos(theta) * L)
45 Theta;
46 int
47 // holds the N value or CurrDepth
48 PTree_N,
49 // holds the base Depth or base N

```

```

50 | base_N,
51 | // holds the number of leaves
52 | leaf_count;
53 | // draws the tree
54 | Color
55 | Skin,
56 | Bark;
57 | vector<RectangleShape> Rects;
58 | virtual void draw(RenderTarget&, RenderStates) const;
59 |
60 | public:
61 | // default constructor
62 | PTree();
63 | // two parameter constructor
64 | PTree(int, double);
65 | // three parameter constructor
66 | PTree(int, double, double);
67 | // gets the base length value
68 | double getBaseLength() { return this->base_L; }
69 | // gets the Theta value
70 | double getTheta() { return this->Theta; }
71 | // gets the initial base length or first length
72 | int getBaseDepth() { return this->base_N; }
73 | // gets the current iteration of the whole tree
74 | int getCurrDepth() { return this->PTree_N; }
75 | // gets the number of leaves in the tree
76 | int getLeafCount() { return this->leaf_count; }
77 | // gets the number of branches and leaves [&RectangleShape]
   |     size
78 | int getSize() { return static_cast<int>(this->Rects.size());
   |     }
79 | // sets N value or CurrDepth
80 | void setCurrDepth(int x) { this->PTree_N = x; }
81 | // sets Theta value
82 | void setTheta(double x) { this->Theta = x; }
83 | // converts double to radians
84 | double DToR(double degrees) { return (degrees * (M_PI / 180)
   |     ); }
85 | // converts radians to a double
86 | double RToD(double radian) { return (radian * (180.0 / M_PI)
   |     ); }
87 |
88 | // insertion op overload
89 | friend ostream& operator<<(ostream&, PTree&);
90 | // extraction op overload
91 | friend istream& operator>>(istream&, PTree&);
92 |
93 | void pTree(int, const RectangleShape&);
94 | };
95 | #endif // PTREE_H_

```



```

1 // Copyright Matthew Boiarski 2023
2 #include "PTree.h"
3 #include <SFML/Graphics/RectangleShape.hpp>
4 int main(int argc, char * argv[]) {
5     int N, len;
6     double L, T;
7     if (argc == 1) cout << "NO PARAMS!\t ENTER YOUR INPUT AS: '
        double, int'\n";
8     if (argc == 2) cout << "ONE PARAM!\t ENTER YOUR INPUT AS: '
        double, int'\n";
9     if (argc == 3) {
10 L = atof(argv[1]);
11 len = static_cast<int>(L);
12 N = atoi(argv[2]);
13 PTree tree(N, L);
14 // cout << tree;
15 RenderWindow window(VideoMode(6 * len, 4 * len), "PTree W/2
        Params");
16 while (window.isOpen()) {
17     Event event;
18     while (window.pollEvent(event)) {
19         if (event.type == Event::Closed)
20             window.close();
21         break;
22     }
23     window.clear(Color::White);
24     window.draw(tree);
25     window.display();
26 }
27 }
28 if (argc == 4) {
29 L = atof(argv[1]);
30 len = static_cast<int>(L);
31 N = atoi(argv[2]);
32 T = atof(argv[3]);
33 PTree tree(N, L, T);
34 // cout << tree;
35 RenderWindow window(VideoMode(6 * len, 4 * len), "Pythagoras
        Tree W/3 Params");
36 while (window.isOpen()) {
37     Event event;
38     while (window.pollEvent(event)) {
39         if (event.type == Event::Closed)
40             window.close();
41         break;
42     }
43     window.clear(Color::White);
44     window.draw(tree);
45     window.display();

```

```
46 | }  
47 | }  
48 | return 0;  
49 | }
```

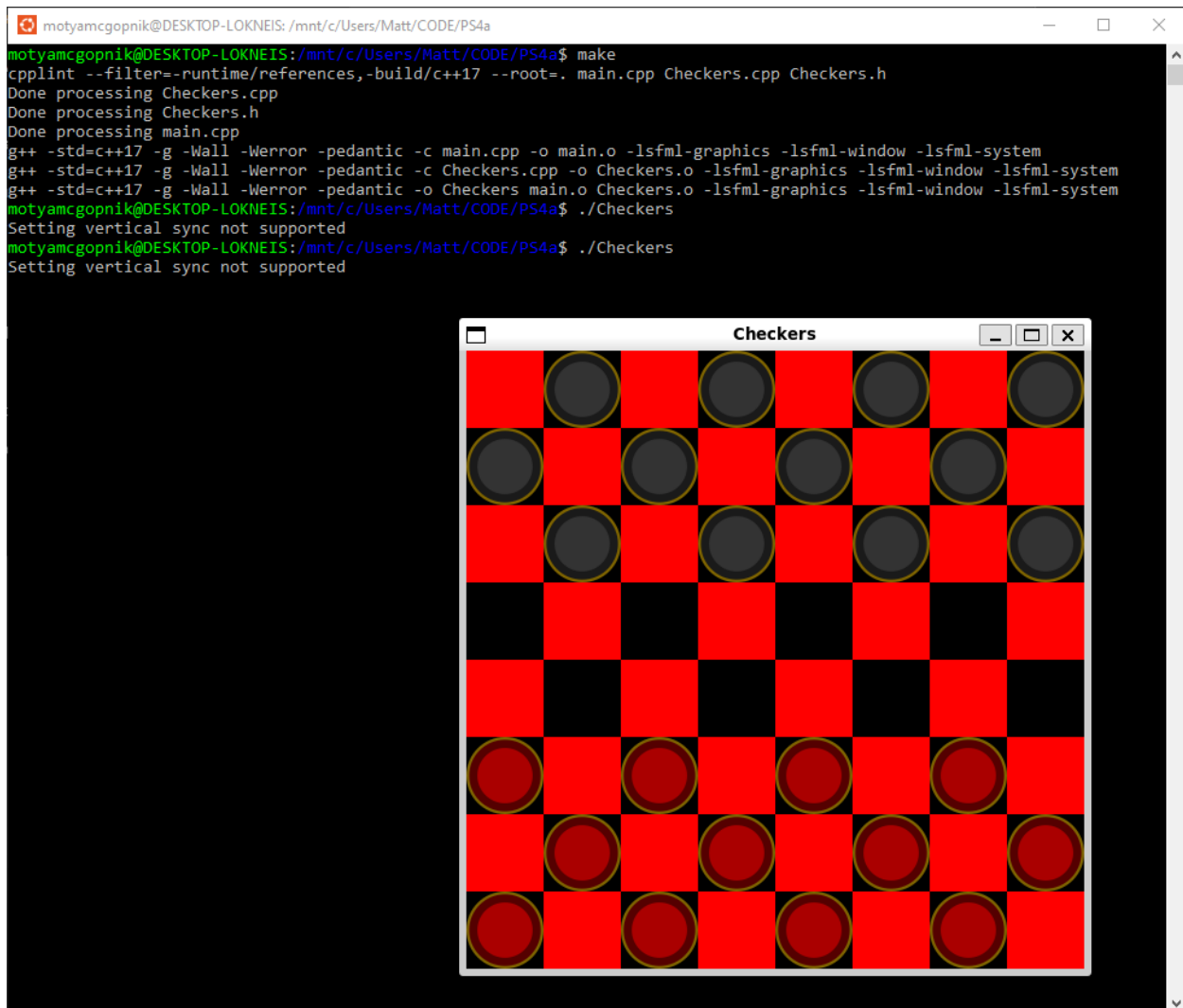
PS4a Checkers Part A

Program Outline: Much like PS2 or the Sokoban game, I am to make a basic setup and highlighting feature for a Checkers game in SFML. I used a 2d char array to hold the gameboard, and `sf::RectangleShapes` for the black and red squares, and the yellow transparent one for selection, as well as a `sf::Sprite` for each game piece.

I learned: How to easily pass in a gameboard with a “staggered” or $\text{mod } 2$ dependent checkerboard layout for the background of the gameboard itself, and how to efficiently set everything using loops.

I did this as one of the assignments in the second PSX assignment correctly.

Screenshot of Output:



```
1: CC = g++
2: CFLAGS = -std=c++17 -g -Wall -Werror -pedantic
3: SFMLFLAGS = -lsfml-graphics -lsfml-window -lsfml-system
4:
5: all: lint Checkers Checkers.o main.o
6:
7: Checkers: main.o Checkers.o
8:     $(CC) $(CFLAGS) -o Checkers main.o Checkers.o $(SFMLFLAGS)
9: main.o: main.cpp Checkers.h
10:     $(CC) $(CFLAGS) -c main.cpp -o main.o $(SFMLFLAGS)
11: Checkers.o: Checkers.cpp Checkers.h
12:     $(CC) $(CFLAGS) -c Checkers.cpp -o Checkers.o $(SFMLFLAGS)
13: lint:
14:     cpplint --filter=--runtime/references,-build/c++17 --root=. main.c
pp Checkers.cpp Checkers.h
15: clean:
16:     rm *.o
17:     rm Checkers
```

```

1 // Copyright Matthew Boiarski 2023
2 #include "Checkers.h"
3
4 int main(int argc, char* argv[]) {
5     RenderWindow gameWindow(VideoMode(WINDOW, WINDOW),
6         "Checkers", sf::Style::Titlebar | sf::Style::Close);
7     Event event;
8     bool isSelect = false;
9     Checkers game;
10    RectangleShape rec;
11    Vector2f hold(PIXEL_CONVERT, PIXEL_CONVERT);
12    while (gameWindow.isOpen()) {
13        while (gameWindow.pollEvent(event)) {
14            if (event.type == Event::Closed) {
15                gameWindow.close();
16            } else if (event.type == Event::MouseButtonPressed) {
17                game.select(event.mouseButton.x / PIXEL_CONVERT,
18                    event.mouseButton.y / PIXEL_CONVERT);
19                if (game.getLocal_it() != -1) {
20                    if (isSelect) {
21                        rec.setFillColor(Color(255, 255, 0, 5));
22                        rec.setPosition((game.getLocal_it() % 8) *
23                            PIXEL_CONVERT,
24                            (game.getLocal_it() / 8) * PIXEL_CONVERT)
25                        ;
26                        rec.setSize(Vector2f(PIXEL_CONVERT,
27                            PIXEL_CONVERT));
28                        gameWindow.draw(rec);
29                        isSelect = true;
30                    }
31                    if (isSelect) {
32                        rec.setFillColor(Color::Black);
33                        rec.setPosition((game.getLocal_it() % 8) *
34                            PIXEL_CONVERT,
35                            (game.getLocal_it() / 8) * PIXEL_CONVERT)
36                        ;
37                        rec.setSize(Vector2f(PIXEL_CONVERT,
38                            PIXEL_CONVERT));
39                        gameWindow.draw(rec);
40                        isSelect = false;
41                    }
42                }
43            }
44        }
45        gameWindow.clear(Color::White);
46        gameWindow.draw(game);
47        gameWindow.display();
48    }
49 }

```

```

1  // Copyright Matthew Boiarski 2023
2  #ifndef CHECKERS_H_
3  #define CHECKERS_H_
4  #include <SFML/System.hpp>
5  #include <SFML/Window.hpp>
6  #include <SFML/Graphics.hpp>
7  #define WINDOW 512
8  #define PIXEL_CONVERT (WINDOW / 8)
9
10
11 #define RED_P 'r'
12 #define RED_K 'R'
13 #define BLACK_P 'b'
14 #define BLACK_K 'B'
15 #define EMPTY '#'
16
17
18 using sf::Drawable;
19 using sf::Color;
20 using sf::RectangleShape;
21 using sf::Texture;
22 using sf::Sprite;
23 using sf::RenderTarget;
24 using sf::RenderStates;
25 using sf::RenderWindow;
26 using sf::VideoMode;
27 using sf::Event;
28 using sf::Vector2f;
29
30 class Checkers : public Drawable {
31 private:
32     Texture
33         RED_PAWN,
34         RED_KING,
35         BLACK_PAWN,
36         BLACK_KING;
37     RectangleShape
38         SELECT;
39     char gameboard[8][8];
40     int
41         num_red_pawns,
42         num_red_kings,
43         num_black_pawns,
44         num_black_kings,
45         local_it;
46     virtual void draw(RenderTarget&, RenderStates) const;
47 public:
48     Checkers();
49     int getLocal_it() { return local_it; }

```

```
50     void select(int, int);  
51 };  
52 #endif // CHECKERS_H_
```



```

1 // Copyright Matthew Boiarski 2023
2 #include "Checkers.h"
3
4 Checkers::Checkers() {
5     RED_PAWN.loadFromFile("./c_imgs/redpawn.png");
6     RED_KING.loadFromFile("./c_imgs/redking.png");
7     BLACK_PAWN.loadFromFile("./c_imgs/blackpawn.png");
8     BLACK_KING.loadFromFile("./c_imgs/blackking.png");
9
10    for (int x = 0; x < 8; x++) {
11        for (int y = 0; y < 8; y++) {
12            if ((x + y) % 2 == 1 && y < 3) gameboard[x][y] =
                BLACK_P;
13            else if (((x + y) % 2 == 1) && y > 4) gameboard[
                x][y] = RED_P;
14            else
15                gameboard[x][y] = EMPTY;
16        }
17    }
18    this->local_it = -1;
19    this->num_red_pawns = 12;
20    this->num_black_pawns = 12;
21    this->num_red_kings = 0;
22    this->num_black_kings = 0;
23 }
24
25 void Checkers::draw(RenderTarget& target, RenderStates
    states) const {
26     RectangleShape rec;
27     Sprite spr;
28     Vector2f hold(PIXEL_CONVERT, PIXEL_CONVERT);
29     for (int row = 0; row < 8; row++) {
30         for (int col = 0; col < 8; col++) {
31             if ((row + col) % 2 == 0) { rec.setFillColor(
                Color::Red);
32             } else {
33                 rec.setFillColor(Color::Black);
34             }
35             rec.setPosition(row * PIXEL_CONVERT, col *
                PIXEL_CONVERT);
36             rec.setSize(Vector2f(PIXEL_CONVERT,
                PIXEL_CONVERT));
37             target.draw(rec);
38             if (gameboard[row][col] != EMPTY) {
39                 if (gameboard[row][col] == RED_P) {
40                     spr.setTexture(RED_PAWN);
41                 }
42                 if (gameboard[row][col] == RED_K) {
43                     spr.setTexture(RED_KING);

```

```

44         }
45         if (gameboard[row][col] == BLACK_P) {
46             spr.setTexture(BLACK_PAWN);
47         }
48         if (gameboard[row][col] == BLACK_K) {
49             spr.setTexture(BLACK_KING);
50         }
51         spr.setPosition(row * PIXEL_CONVERT, col *
52             PIXEL_CONVERT);
53         spr.setScale(hold.x / spr.getLocalBounds().
54             width,
55             hold.y / spr.getLocalBounds().height);
56         target.draw(spr);
57     }
58     if (local_it != -1) {
59         rec.setFill(Color(255, 255, 0, 5));
60         rec.setPosition((local_it % 8) *
61             PIXEL_CONVERT,
62             (local_it / 8) * PIXEL_CONVERT);
63         rec.setSize(Vector2f(PIXEL_CONVERT,
64             PIXEL_CONVERT));
65         target.draw(rec);
66     }
67 }
68
69 void Checkers::select(int x, int y) {
70     if (gameboard[x][y] != EMPTY)
71         local_it = x + y * 8;
72 }

```

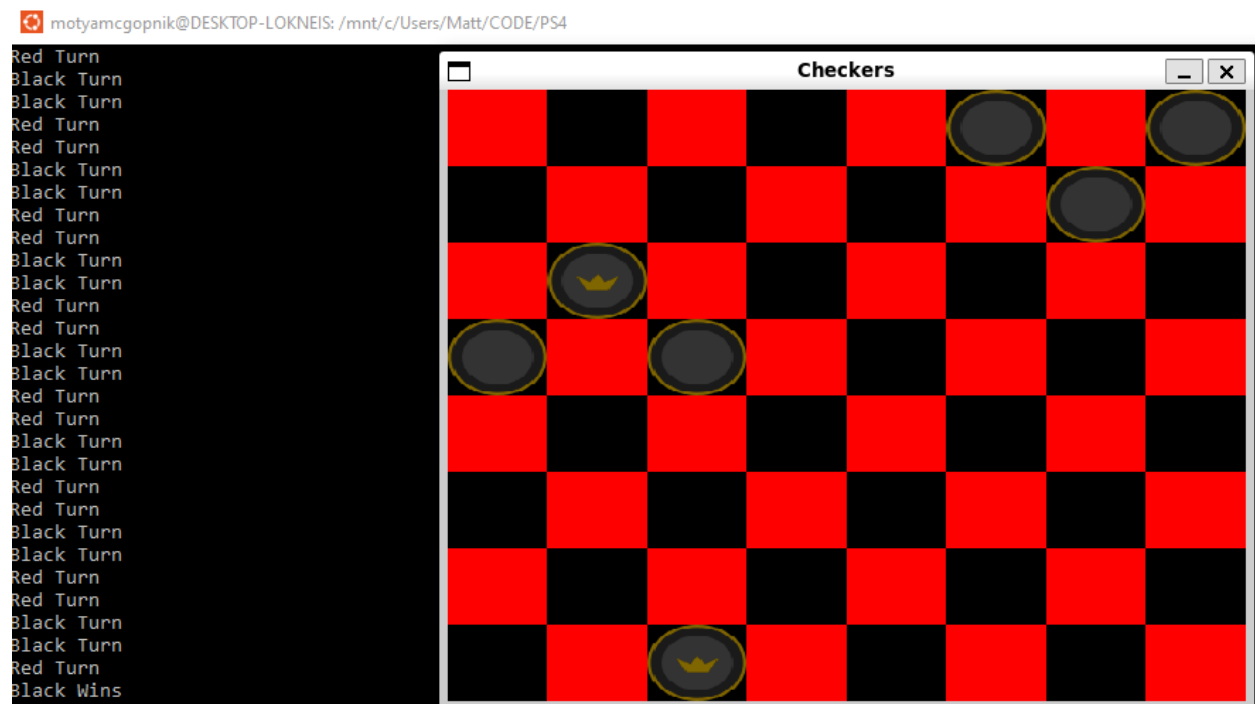
PS4b Checkers Part B

Program Outline: Now that I have a part A to go off of, I now implemented game rules, which limit movement to the black squares only. On top of that, there needs to be a mandatory switching of player's turns back and forth. I also solved the extra credit which forced the player to do multiple jumps if they exist in this gameboard state.

I learned: As a game rule needs to be implemented, I realized that on a larger scale, there is a lot more to do with possibly even regexs' and the way to see if they are valid or contain the right data and in the right order.

I did this as one of the assignments in the second PSX assignment correctly.

Screenshot of Output:



```
1: CC = g++
2: CFLAGS = -std=c++17 -g -Wall -Werror -pedantic
3: SFMLFLAGS = -lsfml-graphics -lsfml-window -lsfml-system
4:
5: all: lint Checkers Checkers.o main.o
6:
7: Checkers: main.o Checkers.o
8:     $(CC) $(CFLAGS) -o Checkers main.o Checkers.o $(SFMLFLAGS)
9: main.o: main.cpp Checkers.h
10:     $(CC) $(CFLAGS) -c main.cpp -o main.o $(SFMLFLAGS)
11: Checkers.o: Checkers.cpp Checkers.h
12:     $(CC) $(CFLAGS) -c Checkers.cpp -o Checkers.o $(SFMLFLAGS)
13: lint:
14:     cpplint --filter=--runtime/references,-build/c++17 --root=. main.c
pp Checkers.cpp Checkers.h
15: clean:
16:     rm *.o
17:     rm Checkers
```

```

1 // Copyright Matthew Boiarski 2023
2
3 #include "Checkers.h"
4 #include <SFML/System.hpp>
5 #include <SFML/Window.hpp>
6 #include <SFML/Graphics.hpp>
7
8
9 int main(int argc, char* argv[]) {
10     RenderWindow gameWindow(VideoMode(WINSIZE, WINSIZE),
11                             "Checkers", sf::Style::Titlebar | sf::Style::Close);
12     sf::Event event;
13
14     Checkers c;
15
16     while (gameWindow.isOpen()) {
17         while (gameWindow.pollEvent(event)) {
18             if (event.type == Event::Closed) {
19                 gameWindow.close();
20             }
21             if (event.type == Event::MouseButtonPressed) {
22                 int mX = event.mouseButton.x / PIXELSPACE;
23                 int mY = event.mouseButton.y / PIXELSPACE;
24                 c.selectMove(mX, mY);
25             }
26         }
27         gameWindow.clear(Color::White);
28         gameWindow.draw(c);
29         gameWindow.display();
30     }
31 }

```

```

1  // Copyright Matthew Boiarski 2023
2  #include <iostream>
3  #include <string>
4  #include <algorithm>
5  #include <numeric>
6  #include <cmath>
7  #include <SFML/Window.hpp>
8  #include <SFML/Graphics.hpp>
9  #define WINSIZE 800
10 #define PIXELSPACE (WINSIZE / 8)
11
12 #define BP 'b'
13 #define BK 'B'
14 #define RP 'r'
15 #define RK 'R'
16 using std::cout;
17 using std::endl;
18 using std::string;
19 using sf::Drawable;
20 using sf::RectangleShape;
21 using sf::Sprite;
22 using sf::Texture;
23 using sf::RenderTarget;
24 using sf::RenderStates;
25 using sf::RenderWindow;
26 using sf::VideoMode;
27 using sf::Event;
28 using sf::Color;
29 using sf::Vector2f;
30 class Checkers : public Drawable {
31 private:
32     Texture rPawn, rKing, bPawn, bKing;
33     bool gameOver = false;
34     string congratulations = "Congrats on your victory";
35     bool blackTurn = true;
36     char board[8][8];
37     bool forceAttack = false;
38     int selectedPiece;
39     bool validMove(int, int, int, int);
40     virtual void draw(RenderTarget&, RenderStates) const;
41 public:
42     Checkers();
43     void selectPiece(int, int);
44 };

```

```

1 // Copyright Matthew Boiarski 2023
2
3 #include "Checkers.h"
4
5 Checkers::Checkers() {
6     RP.loadFromFile("./c_imgs/redpawn.png");
7     RK.loadFromFile("./c_imgs/redking.png");
8     BP.loadFromFile("./c_imgs/blackpawn.png");
9     BK.loadFromFile("./c_imgs/blackking.png");
10    // load in pieces
11    for (int i = 0; i < 8; i++) {
12        for (int j = 0; j < 8; j++) {
13            if ((i + j) % 2 == 1 && j < 3) gameBoard[i][j] =
                BLACK_PAWN;
14            else if (((i + j) % 2 == 1) && j > 4) gameBoard[i][j]
                = RED_PAWN;
15            else
16                gameBoard[i][j] = 0;
17        }
18    }
19    select = -1;
20    gameStatus = false;
21    killBit = false;
22    turn = 1;
23 }
24
25 void Checkers::draw(RenderTarget& target, RenderStates
    states) const {
26     RectangleShape square;
27     Sprite piece;
28     Vector2f res(PIXELSPACE, PIXELSPACE);
29     for (int i = 0; i < 8; i++) {
30         for (int j = 0; j < 8; j++) {
31             if ((i + j) % 2 == 0) {
32                 square.setFillColor(Color::Red);
33             } else { square.setFillColor(Color::Black); }
34             square.setPosition(i * PIXELSPACE, j * PIXELSPACE);
35             square.setSize(Vector2f(PIXELSPACE, PIXELSPACE));
36             target.draw(square);
37             if (gameBoard[i][j] != 0) {
38                 switch (gameBoard[i][j]) {
39                     case RED_PAWN:
40                         piece.setTexture(RP);
41                         break;
42                     case RED_KING:
43                         piece.setTexture(RK);
44                         break;
45                     case BLACK_PAWN:
46                         piece.setTexture(BP);

```

```

47         break;
48     case BLACK_KING:
49         piece.setTexture(BK);
50         break;
51     }
52     piece.setPosition(i * PIXELSPACE, j * PIXELSPACE);
53     piece.setScale(res.x / piece.getLocalBounds().width
54         ,
55         res.y / piece.getLocalBounds().height);
56     target.draw(piece);
57 }
58 if (select != -1) {
59     square.setFill(Color(255, 255, 0, 15));
60     square.setPosition((select % 8) * PIXELSPACE,
61         (select / 8) * PIXELSPACE);
62     square.setSize(Vector2f(PIXELSPACE, PIXELSPACE));
63     target.draw(square);
64 }
65 }
66 }
67
68 bool Checkers::isMove(int x, int y, int pickX, int pickY) {
69     int modX_2 = pickX + (x / 2);
70     int modY_2 = pickY + (y / 2);
71     if (gameBoard[pickX][pickY] == 0) return false;
72     if (pickX + x > 7 || pickY + y > 7) return false;
73     if (pickX + x < 0 || pickY + y < 0) return false;
74     if ((gameBoard[pickX][pickY] == BLACK_KING ||
75         gameBoard[pickX][pickY] == BLACK_PAWN) && !turn)
76         return false;
77     if ((gameBoard[pickX][pickY] == RED_KING ||
78         gameBoard[pickX][pickY] == RED_PAWN) && turn) return
79         false;
80     if (y < 0 && gameBoard[pickX][pickY] == BLACK_PAWN) return
81         false;
82     if (y > 0 && gameBoard[pickX][pickY] == RED_PAWN) return
83         false;
84     if (abs(x) > 2 || abs(y) > 2 || abs(x) != abs(y)) return
85         false;
86     if (abs(x) == 1) if (gameBoard[pickX + x][pickY + y] == 0)
87         return true;
88     if (abs(x) == 2 && gameBoard[pickX + x][pickY + y] == 0) {
89         if (gameBoard[modX_2][modY_2] > 2 && turn) return true;
90         if (gameBoard[modX_2][modY_2] < 3
91             && gameBoard[modX_2][modY_2] != 0 && !turn) return true;
92     }
93     return false;
94 }
95 }

```



```

90 void Checkers::selectMove(int x, int y) {
91     if (gameStatus) return;
92     int pickX = select % 8;
93     int pickY = select / 8;
94     int xMPX = x - pickX;
95     int yMY = y - pickY;
96     if (killBit) {
97         if (abs(x - pickX) != 2) return;
98         if (isMove(xMPX, yMY, pickX, pickY)) {
99             gameBoard[x][y] = gameBoard[pickX][pickY];
100             gameBoard[pickX][pickY] = 0;
101             gameBoard[pickX + (xMPX / 2)][pickY + (yMY / 2)] =
                0;
102             select = x + y * 8;
103         if (y == 7 && turn) gameBoard[x][y] = BLACK_KING;
104         if (y == 0 && !turn) gameBoard[x][y] = RED_KING;
105         pickX = x;
106         pickY = y;
107         select = x + y * 8;
108     }
109     if (isMove(2, 2, pickX, pickY) || isMove(2, -2, pickX, pickY
        )
110     || isMove(-2, 2, pickX, pickY) || isMove(-2, -2, pickX,
        pickY)) {
111         return;
112     }
113
114     killBit = false;
115     (turn) ? turn = 0 : turn = 1;
116     select = -1;
117 }
118 } else if (gameBoard[x][y] == ((turn) ? BLACK_PAWN :
        RED_PAWN)
119     || gameBoard[x][y] == ((turn) ? BLACK_KING : RED_KING)) {
120     select = x + y * 8;
121 } else if (select != -1) {
122     if (y > pickY && gameBoard[pickX][pickY] == RED_PAWN) return
        ;
123     if (y < pickY && gameBoard[pickX][pickY] == BLACK_PAWN)
        return;
124     if (isMove(x - pickX, y - pickY, pickX, pickY)) {
125         gameBoard[x][y] = gameBoard[pickX][pickY];
126         gameBoard[pickX][pickY] = 0;
127         if (y == 7 && turn) gameBoard[x][y] = BLACK_KING;
128         if (y == 0 && !turn) gameBoard[x][y] = RED_KING;
129         if (abs(x - pickX) == 2) {
130             gameBoard[pickX + (xMPX / 2)][pickY + (yMY / 2)] = 0;
131             pickX = x;
132             pickY = y;
133             select = pickX + pickY * 8;

```

```

134 | if (isMove(2, 2, pickX, pickY) || isMove(2, -2, pickX, pickY
    | )
135 | || isMove(-2, 2, pickX, pickY) || isMove(-2, -2, pickX,
    | pickY)) {
136 | killBit = true;
137 | return;
138 | }
139 | }
140 | (turn) ? turn = 0 : turn = 1;
141 | select = -1;
142 | }
143 | }
144 | cout << ((turn) ? "Black Turn" : "Red Turn") << endl;
145 | bool moves = false;
146 | for (int x = 0; x < 8; x++) {
147 | for (int y = 0; y < 8; y++) {
148 | if (isMove(1, 1, x, y) || isMove(-1, 1, x, y)
149 | || isMove(1, -1, x, y) || isMove(-1, -1, x, y))
150 | moves = true;
151 | if (isMove(2, 2, x, y) || isMove(-2, 2, x, y) ||
152 | isMove(2, -2, x, y) || isMove(-2, -2, x, y))
153 | moves = true;
154 | }
155 | }
156 | if (!moves) {
157 | cout << ((turn) ? "Red Wins" : "Black Wins") << endl;
158 | gameStatus = true;
159 | }
160 | }

```

PS5 DNA Sequence Alignment

Program Outline:

I used a Needleman-Wunsch matrix to solve the edit-distance problem. I am given DNA strands that need to be compared to each other in the most simple way possible or with the lowest possible edit distance. If an element in either strand doesn't match at that position, then there is a penalty of 1, if the strand is shorter than the other, then empty spaces compared with a character, then there is a penalty of 2, else if they match then no penalty or 0. After I was to display the strands.

I learned: How to use a dynamic programming approach to a seemingly complex task to decompose it into easier work.

Screenshot of Output:

```
root@LAPTOP-WORKSTATION:/mnt/c/Users/Matt/ps5# ./EDistance < sequence/ecoli2500.txt
Please enter your first DNA segment: Please enter your second DNA segment: Edit distance = 118
```

// Displaying strand is too long here, so it is cut off at the edit distance.

```
root@LAPTOP-WORKSTATION:/mnt/c/Users/Matt/ps5# ./EDistance
Please enter your first DNA segment: ADCADDCCA
Please enter your second DNA segment: ACAADDACC
Edit distance = 4
A A 0
C D 1
A C 1
A A 0
D D 0
D D 0
A C 1
C C 0
C A 1
Elapsed time : 0.000173 seconds.
```

```
1: CC = g++
2: CFLAGS = --std=c++17 -g -Wall -Werror -pedantic
3: LIB = -lsfml-system -lboost_unit_test_framework
4: OBJS = EDistance.o main.o
5:
6: .PHONY: all clean lint
7:
8: all: EDistance
9:
10: EDistance: $(OBJS)
11:      $(CC) $(CFLAGS) -o $@ $^ $(LIB)
12:
13: %.o: %.cpp $(DEPS)
14:      $(CC) $(CFLAGS) -c $<
15:
16: clean:
17:      rm *.o
18:      rm EDistance
19:
20: lint:
21:      cpplint *.cpp *.hpp
```

```

1 // Copyright Matthew Boiarski 2023
2 #include "EDistance.hpp"
3 #include <SFML/System.hpp>
4
5 int main(int argc, char* argv[]) {
6     string first, second, tmp;
7     cout << "Please enter your first DNA segment: ";
8     cin >> first;
9     cout << "Please enter your second DNA segment: ";
10    cin >> second;
11    if (first.length() > second.length()) {
12        tmp = first;
13        first = second;
14        second = tmp;
15    }
16    sf::Clock c;
17    EDistance arr(first, second);
18    cout << "Edit distance = " << arr.getEditDistance() <<
        endl;
19    tmp = arr.alignment();
20    int i = 0;
21    for_each(tmp.begin(), tmp.end(), [second, &i](char c) {
22        cout << second[i] << ", " << c << endl;
23        i++;
24    });
25    cout << "Elapsed time : " <<
26    c.getElapsedTime().asSeconds() << " seconds." << endl;
27    return 0;
28 }

```

```

1 // Copyright Matthew Boiarski 2023
2 #ifndef EDISTANCE_HPP_
3 #define EDISTANCE_HPP_
4 #include <iostream>
5 #include <iomanip>
6 #include <string>
7 #include <vector>
8 #include <algorithm>
9 using std::cin;
10 using std::cout;
11 using std::endl;
12 using std::string;
13 using std::vector;
14 class EDistance {
15 private:
16 vector <int> plot;
17 string DNA1, DNA2;
18 int
19     N, M,
20     editDistance;
21 int vPush(int, int);
22 static int penalty(char a, char b);
23 static int min(int a, int b, int c);
24 public:
25 EDistance(string, string);
26 int getEditDistance() {
27 editDistance = optDistance();
28 return editDistance;
29 }
30 int optDistance();
31 string alignment();
32 };
33 #endif // EDISTANCE_HPP_

```

```

1 // Copyright Matthew Boiarski 2023
2 #include "EDistance.hpp"
3
4 // returns a value that is the penalty charge between both
   characters
5 int EDistance::penalty(char a, char b) {
6     if (a == b) return 0;
7     return 1;
8 }
9
10 // returns the minimum value
11 int EDistance::min(int a, int b, int c) {
12     int res = (a > b) ? b : a;
13     res = (res > c) ? c : res;
14     return res;
15 }
16 // shoves values into a single vector that is i * j * (N +
   1) deep for this value
17 int EDistance::vPush(int i, int j) {
18     return i + j * (N + 1);
19 }
20
21 EDistance::EDistance(string a, string b) {
22     DNA1 = a;
23     DNA2 = b;
24     N = (int)a.length();
25     M = (int)(b.length());
26     plot = vector<int>((N + 1) * (M + 1));
27     editDistance = -1;
28 }
29
30
31 int EDistance::optDistance() {
32     for (int i = M; i >= 0; i--) {
33         plot[vPush(N, i)] = (M - i) * 2;
34     }
35     for (int i = N; i >= 0; i--) {
36         plot[vPush(i, M)] = (N - i) * 2;
37     }
38     for (int i = N - 1; i >= 0; i--) {
39         for (int i2 = M - 1; i2 >= 0; i2--) {
40             int bitOne = plot[vPush(i + 1, i2 + 1)] + penalty(DNA1[i],
               DNA2[i2]);
41             int bitTwo = plot[vPush(i + 1, i2)] + 2;
42             int bitThree = plot[vPush(i, i2 + 1)] + 2;
43             plot[vPush(i, i2)] = min(bitOne, bitTwo, bitThree);
44             editDistance = plot[0];
45         }
46     }

```

```

47|     return editDistance;
48| }
49|
50| string EDistance::alignment() {
51|     string ret = "";
52|     int i = 0;
53|     int j = 0;
54|     while (i != N || j != M) {
55|         if (plot[vPush(i, j)] ==
56|             (plot[vPush(i + 1, j + 1)] + penalty(DNA1[i],
57|             DNA2[j]))) {
58|             cout << DNA2[j] << " " << DNA1[i] << " "
59|                 << penalty(DNA1[i], DNA2[j]) << endl;
60|             i++;
61|             j++;
62|         } else if (i < N && plot[vPush(i, j)] ==
63|             (plot[vPush(i + 1, j)] + 2)) {
64|             cout << "-" << DNA1[i] << " 2" << endl;
65|             i++;
66|         } else if (j < M && plot[vPush(i, j)] ==
67|             (plot[vPush(i, j + 1)] + 2)) {
68|             cout << DNA2[j] << " -" << " 2" << endl;
69|             j++;
70|         }
71|     }
72|     return ret;

```


Program Outline: In this program, I needed to take a string, make a kgram out of some selection of said string, and then randomly generate a new string out of only kgrams repeatedly until it reaches the end. I also needed to implement this as a Markov model.

I was supposed to add in a lambda, but in this implementation, I did not.

```
motyamcgopnik@DESKTOP-LOKNEIS:/mnt/c/Users/Matt/CODE/PS6$ ./TextWriter 3 500 < markov/IolantheIlibretto.txt beatles.txt input17.txt trump-clinton1.txt  
aesop.txt bible.txt input53.txt trump-clinton2.txt  
amendments.txt deadend.txt monalisa.txt trump-clinton3.txt  
bbbababababababa.txt dream.txt pearl_jam.txt wiki_100k.txt  
motyamcgopnik@DESKTOP-LOKNEIS:/mnt/c/Users/Matt/CODE/PS6$ ./TextWriter 10 500 < markov/bible.txt  
In the beginning? I the LORD that rendereth recompence upon you none to remain;  
In that night, and have not heard the noise of thunder:  
And call not one of the man's house, into the hand of their God, and the people, and Chub, and the sand for  
the children of Israel, which was for the punishment of his place of the blood of it is burned up with the  
ir hearts to fulfil the lusts of our God upon the earth.  
And they answered him, Two hundred chariots, and his soul within himself; and the name of  
motyamcgopnik@DESKTOP-LOKNEIS:/mnt/c/Users/Matt/CODE/PS6$
```

```
1: CC = g++
2: CFLAGS = -std=c++11 -Wall -Werror -pedantic
3: OBJ = TextWriter.o RandWriter.o test.o
4: LIB = -lboost_unit_test_framework
5: all: TextWriter $(OBJ)
6:
7: TextWriter: $(OBJ)
8:     $(CC) $(CFLAGS) -o TextWriter $(OBJ) $(LIB)
9:
10: TextWriter.o: TextWriter.cpp RandWriter.h
11:     $(CC) $(CFLAGS) -c TextWriter.cpp -o TextWriter.o
12:
13: RandWriter.o: RandWriter.cpp
14:     $(CC) $(CFLAGS) -c RandWriter.cpp -o RandWriter.o
15: test.o: test.cpp
16:     $(CC) $(CFLAGS) -c test.cpp -o test.o $(LIB)
17: lint:
18:     cpplint --filter=--runtime/references,-build/c++11 --root=. TextWr
iter.cpp RandWriter.cpp RandWriter.h
19:
20: clean:
21:     rm TextWriter $(OBJ)
22:
```

```

1 // Copyright Matthew Boiarski 2023
2 #include "RandWriter.h"
3 #include <boost/test/unit_test.hpp>
4
5 // test constructor
6 BOOST_AUTO_TEST_CASE(Constructor) {
7     int k = 2;
8     string test = "gagggagaggcgagaaa";
9     BOOST_REQUIRE(RandWriter(test, k));
10    BOOST_REQUIRE_NO_THROW();
11 };
12 // test orderK
13
14 // test freq(string)
15 BOOST_AUTO_TEST_CASE(FirstFreqFunc) {
16     int k = 2;
17     string test = "gagggagaggcgagaaa";
18     RandWriter obj(test, k);
19     int temp = obj.freq(test);
20     BOOST_REQUIRE_NO_THROW();
21 };
22 // test freq(string, char)
23 BOOST_AUTO_TEST_CASE(SecondFreqFunc) {
24     int k = 2, temp = -1;
25     string test = "gagggagaggcgagaaa";
26     RandWriter obj(test, k);
27     temp = obj.freq(test, 'a');
28     BOOST_REQUIRE(temp != -1);
29     BOOST_REQUIRE_NO_THROW();
30 };
31 // test kRand(string)
32 BOOST_AUTO_TEST_CASE(kRand) {
33     int k = 2, temp = -1;
34     string test = "gagggagaggcgagaaa";
35     RandWriter obj(test, k);
36     int temp = obj.kRand(test);
37     BOOST_REQUIRE(temp != -1);
38     BOOST_REQUIRE_NO_THROW();
39 };
40 // test generate(string, int)
41 BOOST_AUTO_TEST_CASE(kRand) {
42     int k = 2;
43     string test = "gagggagaggcgagaaa";
44     RandWriter obj(test, k);
45     string temp = obj.generate(test, 3);
46     BOOST_REQUIRE(!test.empty());
47     BOOST_REQUIRE_NO_THROW();
48 };

```

```

1 // Copyright Matthew Boiarski 2023
2 #include "RandWriter.h"
3
4
5 int main(int argc, char* argv[]) {
6     int k = stoi(argv[1]);
7     int l = stoi(argv[2]);
8
9     string str, in;
10
11     while (getline(cin, in)) {
12         str += in + '\n';
13     }
14     RandWriter mark(str, k);
15     string kgram = str.substr(0, k);
16     string made = mark.generate(kgram, l);
17
18     cout << made << endl;
19     return 0;
20 }

```

```

1 // Copyright Matthew Boiarski 2023
2 #ifndef RANDWRITER_H_
3 #define RANDWRITER_H_
4 #include <iostream>
5 #include <cstdlib>
6 #include <string>
7 #include <map>
8 #include <random>
9 #include <stdexcept>
10 using std::map;
11 using std::string;
12 using std::cin;
13 using std::cout;
14 using std::endl;
15 using std::stoi;
16 using std::invalid_argument;
17
18 class RandWriter {
19 public:
20   RandWriter(string, int);
21   int orderK() const;
22   int freq(string) const;
23   int freq(string, char) const;
24   char kRand(string);
25   string generate(string, int);
26
27 private:
28   int order;
29   map<string, int> kgram_i;
30   map<string, map<char, int>> kgram_c_i;
31   map<string, map<double, char>> kgram_d_c;
32 };
33 #endif // RANDWRITER_H_

```

```

1 // Copyright Matthew Boiarski 2023
2 #include "RandWriter.h"
3
4 RandWriter::RandWriter(string text, int k) {
5     if (text.empty()) throw invalid_argument("empty string!");
6     order = k;
7     srand(time(NULL));
8     int len = static_cast<int>(text.length());
9     for (int i = 0; i < len; i++) {
10         string substr;
11         (i + k > len) ?
12             substr = text.substr(i, len - i) + text.substr
13                 (0, k - (len - i)):
14             substr = text.substr(i, k);
15
16         char c = text.at((i + k) % len);
17
18         if (kgram_i.find(substr) != kgram_i.end()) {
19             kgram_i[substr]++;
20             (kgram_c_i[substr].find(c) != kgram_c_i[substr].
21                 end()) ?
22                 kgram_c_i[substr][c]++ : kgram_c_i[substr][c]
23                 = 1;
24         } else {
25             kgram_i[substr] = 1; kgram_c_i[substr] = { {c,
26                 1} };
27         }
28     }
29
30     for (const auto& kgram_freq : kgram_i) {
31         kgram_d_c[kgram_freq.first] = {};
32         int accum_freq = 0;
33         for (const auto char_freq : kgram_c_i[kgram_freq.
34             first]) {
35             accum_freq += char_freq.second;
36             kgram_d_c[kgram_freq.first][accum_freq] =
37                 char_freq.first;
38         }
39     }
40 }
41
42 int RandWriter::orderK() const { return order; }
43
44 int RandWriter::freq(string kgram) const {
45     if (static_cast<int>(kgram.length()) != order) {
46         throw invalid_argument("invalid length!");
47     }
48 }

```

```

43     auto i = kgram_i.find(kgram);
44     if (i == kgram_i.end()) {
45         return 0;
46     } else {
47         return i->second;
48     }
49 }
50
51 int RandWriter::freq(string kgram, char c) const {
52     if (static_cast<int>(kgram.length()) != order) {
53         throw invalid_argument("invalid length!");
54     }
55
56     auto out = kgram_c_i.find(kgram);
57     if (out == kgram_c_i.end()) {
58         return 0;
59     }
60
61     auto in = out->second.find(c);
62     if (in == out->second.end()) {
63         return 0;
64     }
65
66     return in->second;
67 }
68
69 char RandWriter::kRand(string kgram) {
70     if (static_cast<int>(kgram.length()) != order) {
71         throw invalid_argument("invalid length!");
72     }
73
74     int freq_real = kgram_i.at(kgram);
75     int ran = rand() % freq_real;
76     return static_cast<char>(kgram_d_c.at(kgram).upper_bound
77         (ran)->second);
78 }
79 string RandWriter::generate(string kgram, int length) {
80     string res(kgram);
81
82     for (int i = kgram.length() - order; i < length - order;
83         i++) {
84         string substr = res.substr(i, order);
85         char c = kRand(substr);
86         res += c;
87     }
88
89     return res;
90 }

```

PS7 Kronos – Intro to Regular Expression

Program Outline:

In this program, I was to mimic the behavior of a Kronos InTouch device and use regex's to be able to parse through .tlog files, and then to output a .tlog.rpt file with if a server started / ended, and the timestamps of where it existed in computational time. I used specifically a boost::regex as well as boost::date and boost::ptime

I learned: How a regex works, how to implement one for my needs, as well as being able to use a regex is similar to an overcomplicated scanf() function on steroids. While the last line may have been far from computer-science oriented writing, it gets the point across well.

Screenshot of Output:

 device1_intouch.log.rpt - Notepad

File	Edit	Format	View	Help
435369	2014-Mar-25	19:11:59	success	0-3-3
436500	2014-Mar-25	19:29:59	success	0-2-45
440719	2014-Mar-25	22:01:46	success	0-2-41
440866	2014-Mar-26	12:47:42	success	0-2-47
442094	2014-Mar-26	20:41:34	success	0-2-39
443073	2014-Mar-27	14:09:01	success	0-2-41


```
1: CC = g++
2: CFLAGS = -std=c++11 -Wall -Werror -pedantic
3: LIB = -lboost_regex
4:
5: all: ps7 reader.o
6:
7: ps7: reader.o
8:      $(CC) $(CFLAGS) -o ps7 reader.o $(LIB)
9:
10: reader.o: reader.cpp
11:      $(CC) $(CFLAGS) -c reader.cpp -o reader.o $(LIB)
12: lint:
13:      cpplint --filter=--runtime/references,-build/c++11 --root=. reader
.cpp
14:
15: clean:
16:      rm *.o ps7 *.rpt
```

```

1 // Copyright Matthew Boiarski 2023
2 #include <iostream>
3 #include <fstream>
4 #include <string>
5 #include <regex>
6 #include "boost/regex.hpp"
7 #include "boost/date_time/posix_time/posix_time.hpp"
8
9 #define SUCCESS " success"
10 #define FAILURE " failure"
11
12 using std::string;
13 using std::getline;
14 using std::endl;
15 using std::ifstream;
16 using std::ofstream;
17 using std::regex_match;
18 using boost::regex;
19 using boost::posix_time::ptime;
20 using boost::posix_time::time_duration;
21 using boost::posix_time::time_from_string;
22
23 int main(int argc, char *argv[]) {
24     ifstream file(argv[1]);
25     int lnNumber = 1;
26     bool f_NON_NULL = false;
27     ofstream rptFile(string(argv[1]) + ".rpt", ofstream::out);
28     ptime startT, endT;
29     string line, d;
30     while (getline(file, line)) {
31         if (regex_match(line,
32             regex("([0-9]+)-([0-9]+)-([0-9]+) "
33                 "([0-9]+):([0-9]+):([0-9]+): "
34                 "\\(log.c.166\\) server started.*"))) {
35             if (f_NON_NULL) {
36                 rptFile << FAILURE << endl;
37             }
38
39             f_NON_NULL = true;
40             d = line.substr(0, 19);
41             startT = time_from_string(d);
42             rptFile << lnNumber << " " << startT;
43         }
44
45         if (regex_match(line,
46             regex("([0-9]+)-([0-9]+)-([0-9]+) "
47                 "([0-9]+):([0-9]+):([0-9]+):([0-9]+):INFO:"
48                 "oejs.AbstractConnector:Started SelectChannelConnector@0
49                 .0.0.0:9080.*"))) {

```

```

49 if (f_NON_NULL) {
50     f_NON_NULL = false;
51     d = line.substr(0, 23);
52     endT = time_from_string(d);
53     time_duration elapsed = endT - startT;
54     rptFile << SUCCESS << " "
55     << elapsed.hours() << "-" << elapsed.minutes() <<
56     "-" << elapsed.seconds() << endl;
57 }
58 }
59 lnNumber++;
60 }
61
62 if (f_NON_NULL) {
63     rptFile << FAILURE << endl;
64 }
65
66 file.close();
67 rptFile.close();
68 return 0;
69 }

```